

专题：从Web应用看GUI与性能测试
——结合Selenium、Jmeter、大模型等工具

4 网站测试

网站测试在网页测试的基础之上还包括：

4.1 功能测试；

4.2 性能测试；

4.3 安全性测试；

4.4 兼容性测试

4.1 功能测试

功能测试包括以下内容:

1） 数据库测试

在Web 应用中，最常用的数据库类型是关系型数据库，可以使用SQL 对信息进行处理。常见问题

2） WEB网站特定的功能需求测试

依据需求规格说明书对WEB网站特定的功能需求进行验证。

3） 设计语言测试

不同的Web 设计语言版本的差异可以引起客户端或服务器端严重的问题;

例如使用Java、JavaScript、ActiveX、VBScript或Perl 等开发的应用程序也要在不同的版本上进行验证。

4.2 性能测试

包括以下内容:

① 连接速度测试

② 负载测试

③ 压力测试

① 连接速度测试

用户连接方式的不同：都有哪些？

- 电话拨号上网
- 宽带上网
- 局域网（光纤）
- GSM、CDMA、4G、5G
- 无线（WiFi）
- 有线电视网
- 电力网
- 其它？

② 负载测试

负载测试的目的:

- 负载测试是为了测量Web 系统在某一负载级别上的性能，以保证Web 系统在需求范围内能正常工作。

负载测试内容:

- 某个时刻同时访问Web 系统的用户数量;
- 在线数据处理的数量。

例如:

- 系统最多能允许多少个用户同时在线?
- 如果超过了这个数量，会出现什么现象?
- 系统能否处理大量用户同时对同一个页面的请求?

③ 压力测试

压力测试的目的:

压力测试目的是要弄清楚被测试的 Web 服务是不是不仅能做我们认为它能做的事，而且在被施加了某些高强度压力的情况下仍然继续正常运行。

压力测试的内容:

压力测试必须对 Web 服务应用以下四个基本条件进行有效的压力测试。

- 重复（Repetition）；
- 并发（Concurrency）；
- 量级（Magnitude）-（**每个操作中的负载量**）；
- 随机变化。

压力测试的内容

重复（Repetition）：

测试的重复就是一遍又一遍地执行某个操作或功能。比如重复调用一个Web 服务，确定一个操作能否正常执行，并且能否继续在每次执行时都正常。

什么错误会导致重复执行的错误?

压力测试的内容

并发（Concurrency）：

并发是同时执行多个操作的行为。换句话说，就是在同一时间执行多个测试，例如在同一个服务器上同时调用许多 Web 服务。

严格并发如何产生？

压力测试的内容

量级（Magnitude）：

压力测试系统应该应用于产品的另一个条件，需要考虑每个操作中的负载量，即也要尽量给产品增加负担。

例如，改变数据的大小、改变时间延迟的长度、资金数量的转移、输入速度以及输入的变化等。

压力测试的内容

随机变化：

任何压力系统都多多少少具有一些随机性。随机使用前面的压力原则中介绍的无数变化形式，就能够在每次测试运行时应用许多不同的代码路径。

压力测试用例参考模板

1. 被测试对象的介绍
2. 测试范围与目的
3. 测试环境与测试辅助工具的描述
4. 测试驱动程序的设计
5. 压力测试用例

极限名称 A	如“最大并发用户数量”	
前提条件		
输入/动作	输出/响应	是否能正常运行
如 10 个用户并发操作		
如 20 个用户并发操作		

……

4.3 安全性测试

- 登录验证
- 数据加密
- 超时限制
- SSL套接字测试
- 目录测试
- 脚本语言
- 日志文件

Web安全性测试

- 登录验证：一般的网站都会使用登录或者注册后使用的方式，因此，必须对用户名和匹配的密码进行校验，以阻止非法用户登录。在进行登陆测试的时候，需要考虑输入的 密码是否对大小写敏感、是否有长度和条件限制，最多可以尝试多少次登录，哪些页面或者文件 需要登录后才能访问/下载等。

- 数据加密：某些数据需要进行信息加密和过滤后才能进行数据传输，例如用户信用卡信息、用户登陆密码信息等。（包括加密算法、解密算法）

- 超时限制：WEB应用系统需要有是否超时的限制，当用户长时间不作任何操作的时候，需要重新登录才能使用其功能。
- SSL：越来越多的站点使用SSL安全协议进行传送。SSL是Security Socket Lauer(安全套接字协议层)的缩写，是由Netscape首先发表的网络数据安全传输协议。SSL是利用公开密钥/私有密钥的加密技术。（RSA），在位于HTTP层和TCP层之间，建立用户与服务器之间的加密通信，确保所传递信息的安全性。SSL是工作在公共密钥和私人密钥基础上的，任何用户都可以获得公共密钥来加密数据，但解密数据必须要通过相应的私人密钥。

- 目录测试：WEB的目录安全是不容忽视的一个因素。如果WEB程序或WEB服务器的处理不适当，通过简单的URL替换和推测，会将整个WEB目录完全暴露给用户，这样会造成很大的风险和安全性隐患。
- 我们可以使用一定的解决方式，如在每个目录访问时有index.htm,或者严格设定WEB服务器的目录访问权限，将这种隐患降低到最小程度。

- 服务器脚本语言：脚本语言是最常见的安全隐患，如有些脚本语言允许访问根目录，经验丰富的黑客可以通过这些缺陷来攻击和使用服务器系统，因此，脚本语言安全性在测试过程中也必须被考虑到。

- 日志文件：在服务器上，要验证服务器的日志是否正常工作，例如CPU的占用率是否很高，是否有例外的进程占用，所以的事务处理是否被记录等。

4.4 兼容性测试

- 平台测试：操作系统类型，最常见的有Windows、Unix、Macintosh、Linux 、symbian、ios、windows phone等；

- 浏览器测试：不同厂商的浏览器对Java，、JavaScript、ActiveX、plug-ins 或不同的HTML 有不同的支持；
- 常见的有IE、Firfox、Opera、Chrome、360、天天、腾讯TT、傲游、百度等；

- **分辨率测试**：页面版式在不同(640x400、600x800 或1024x768) 分辨率模式下是否显示正常?

常见的显示器分辨率

- **连接速率测试**:不同接入介质的速率测试;

- **组合测试**：前四项的组合。



软件性能测试工具介绍
——JMeter

目录

1. 理解软件性能
2. JMeter简介
3. JMeter测试组件介绍
4. JMeter测试示例
5. 脚本录制工具-Badboy/代理服务器
6. 参考文档

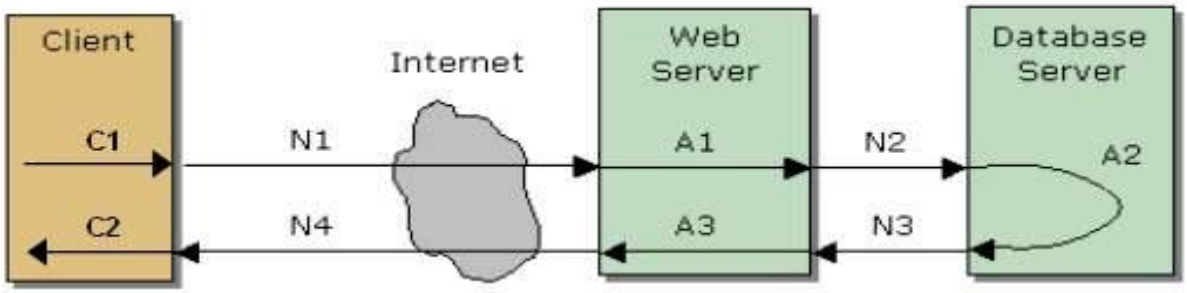
1 理解软件性能

什么是“软件性能”?

- 响应时间
- 并发用户数
- 吞吐量
- 资源可用度
- 其它?

软件性能指标

- **响应时间**



从图中我们可以清楚的看到一个请求的响应时间是由几部分时间组成的，包括

C1：用户请求发出前在客户端需要完成的预处理所需要的时间；

C2：客户端收到服务器返回的响应后，对数据进行处理并呈现所需要的时间；

A1：Web/App Server 对请求进行处理所需要的时间；

A2：DB Server 对请求进行处理所需的时间；

A3：Web/App Server 对 DB Server 返回的结果进行处理所需的时间；

N1：请求由客户端发出并达到Web/App Server 所需要的时间；

N2：如果需要进行**数据库**相关的操作，由Web/App Server 将请求发送至DB Server 所需要的时间；

N3：DB Server 完成处理并将结果返回Web/App Server 所需的时间；

N4：Web/App Server 完成处理并将结果返回给客户端所需的时间；

从用户的角度来看，响应时间 = (C1+C2)+(A1+A2+A3)+(N1+N2+N3+N4)；但是从系统的角度来看，响应时间只包括(A1+A2+A3)+(N1+N2+N3+N4)。

在理解了响应时间的组成之后，可以帮助我们通过对响应时间的分析来更好的识别和定位系统的性能瓶颈。

软件性能指标

并发用户数 v.s. 系统用户数 v.s.同时在线用户数

系统用户数：系统额定的用户数量，如一个OA系统，可能使用该系统的用户总数是2000个，那么这个数量，就是系统用户数。

同时在线用户数：在一定的时间范围内，最大的同时在线用户数量

平均并发用户数的计算：

$$C=nL/T$$

其中C是平均的并发用户数，n是平均每天访问用户数，L是一天内用户从登录到退出的平均时间（操作平均时间），T是考察时间长度（一天内多长时间有用户使用系统）

软件性能指标

吞吐量

在不同的测试工具中，对于吞吐量(Throughput)会有不同的解释。例如，在LoadRunner中，这个指标是以**字节数**为单位来衡量网络吞吐量的，而在JMeter中则是以**事务数/秒**为单位来衡量系统的响应能力的。

不过在大多数英文的性能测试方面的书籍或资料中，吞吐量的定义使用的是后者。

资源可用度

指系统各种资源的使用情况，如cpu占用率、内存占用率等，一般使用“资源实际使用/总的资源可用量”形成资源利用率。

1理解软件性能

用户视角的软件性能

- 从用户的角度来说，软件性能就是软件对用户操作的**响应时间**。说得更明确一点，对用户来说，当用户单击一个按钮、发出一条指令或是在Web页面上单击一个链接，从用户单击开始到应用系统把本次操作的结果以用户能察觉的方式展示出来，这个过程所消耗的时间就是用户对软件性能的直观印象。
- 必须要说明的是，用户所体会到的“响应时间”既有客观的成分，也有主观的成分。例如，用户执行了某个操作，该操作返回大量数据，从客观的角度来说，事务的结束应该是系统返回所有的数据，响应时间应该是从用户操作开始到所有数据返回完成的整个耗时；但从用户的主观感知来说，如果采用一种优化的数据呈现策略，当少部分数据返回之后就立刻将数据呈现在用户面前，则用户感受到的响应时间就会远远小于实际的事务响应时间(顺便说一下，这种技巧是在C／S结构的管理系统中开发人员常用的一种技巧)。

1理解软件性能

管理员视角的软件性能

- 从管理员的角度来看，软件系统的性能首先表现在系统的**响应时间**上，这一点和用户视角是一样的。但管理员是一种特殊的用户，和一般用户相比，除了会关注一般用户的体验之外，他还会关心**和系统状态相关的信息**。
- 另一方面，管理员还会想要知道系统具有多大的**可扩展性**，处理**并发**的能力如何；而且，管理员还会希望知道系统可能的**最大容量**是什么，系统可能的**性能瓶颈**在哪里，通过更换哪些设备或是进行哪些扩展能够提高系统性能，了解这些情况，管理员才能根据系统的用户状况制定管理措施，在系统出现计划之外的用户增长等紧急情况的时候能够立即制定相应措施，进行迅速的处理；此外，管理员可能还会关心系统在长时间的运行中是否足够**稳定**，是否能够不间断地提供业务服务等。
- 因此，从管理员的视角来看，软件性能绝对不仅仅是应用的响应时间这么一个简单的问题。

1理解软件性能

开发者视角的软件性能

- 从开发人员的角度来说，对软件性能的关注就更加深入了。对开发人员来说，其最想知道的是“如何通过**调整设计和代码实现**，或是如何通过**调整系统设置**等方法提高软件的性能表现”，和如何发现并解决软件设计和开发过程中产生的由于多用户访问引起的**缺陷**”。
- 因此，其最关注的是使性能表现不佳的因素和由于大量用户访问引发的软件故障，也就是我们通常所说的“**性能瓶颈**”和系统中存在的在大量用户访问时表现出来的**缺陷**。

2 JMeter简介

- **JMeter**是 Apache 公司基于 Java 开发的一款开源压力测试工具，体积小，功能全，使用方便，是一个轻量级的测试工具，使用起来非常简单。
- 因为 JMeter 是 Java 开发的，所以运行的时候必须先要安装 jdk 才可以。**JMeter 是免安装的，拿到安装包之后直接解压就可以使用**，同时它在 Linux/Windows/MacOS 上都可以使用。
- JMeter 可以做**接口测试和压力测试。**其中接口测试的简单操作包括做**http 脚本**（发 get/post 请求、加 cookie、加 header、加权 限认证、上传文件）、做 webservice 脚本、参数化、断言、关联（正则表达式提取器和处理 json-json path extractor）和 jmeter 操作数据库等。
- 相较于LoadRuneer更加轻量级，易使用

2.1 JMeter下载安装

- 1. http://jmeter.apache.org/下载最新版本的JMeter，解压文件到任意目录
- 2. **配置Java环境**：安装JDK，配置环境变量JAVA_HOME.
- 3. 系统要求：JMeter2.11 需要JDK1.6以上的版本支持运行
- 4. JMeter可以运行在如下操作系统上：Linux，Windows和MacOS
- 5.避免在一个有空格的路径安装JMeter，这将导致远程测试出现问题。

2.2 Jmeter运行

- 1. 进入bin目录运行jmeter.bat启动jmeter
  - 注意：打开的时候会有两个窗口，JMeter的命令窗口和JMeter的图形操作界面，不可以关闭命令窗口。
- 2. JMeter的classpath:
  - 如果你想添加其他JAR文件到JMeter的classpath中，你必须复制他们到lib目录中;
  - 如果你开发了一个JMeter特定组件或有效的jar文件，复制到lib目录下的ext目录中。
- 3. 打开之后显示的是中文，如果你想使用其他语言，比如英文，那么通过菜单选项->选择语言->英文即可，当然转为中文也是同样操作。

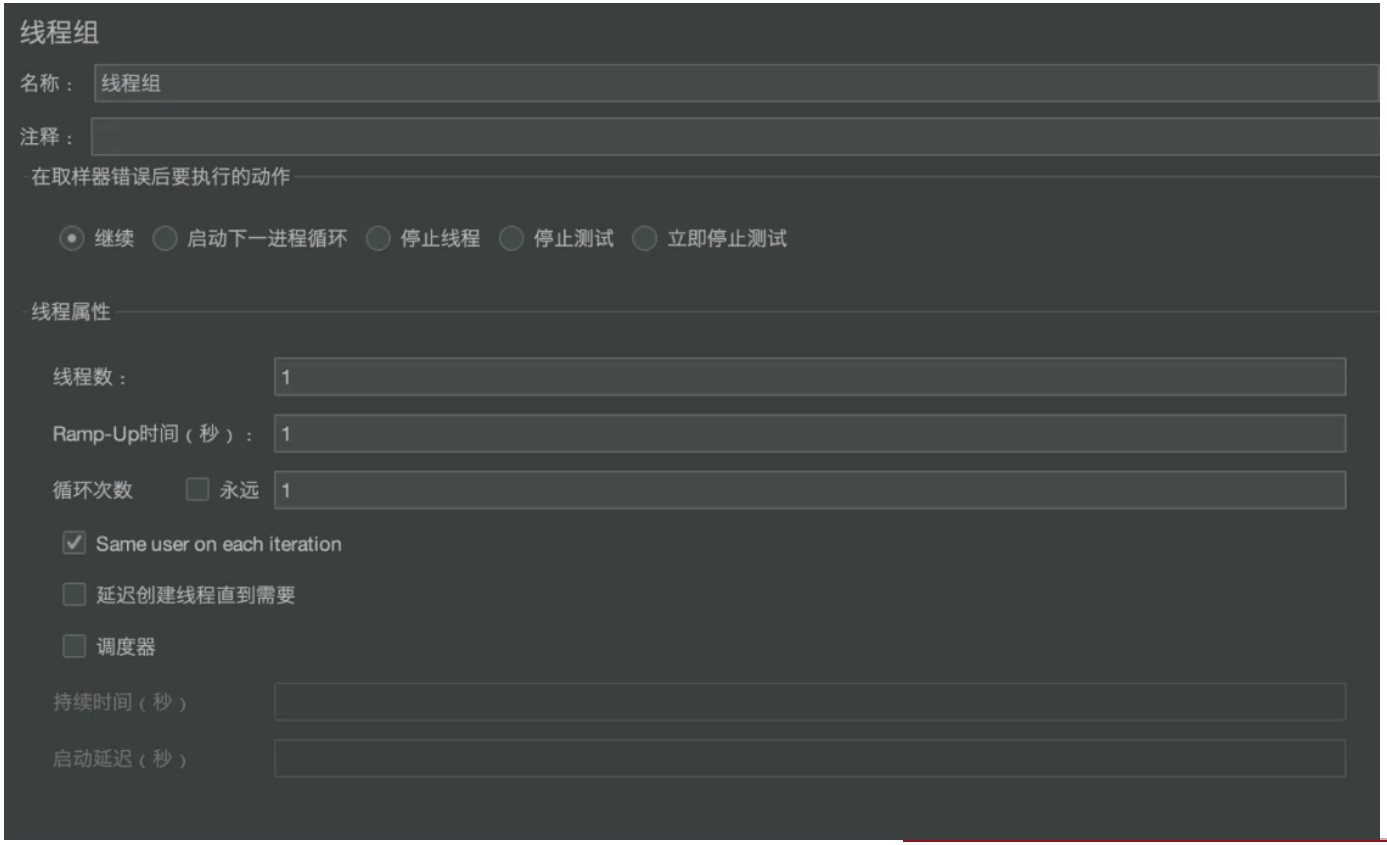
3 JMeter测试组件介绍

- JMeter的测试脚本在界面中以「树」的形式呈现，保存后的测试脚本jmx文件本身也是xml格式。JMeter脚本树以测试计划(Test Plan)为根节点，所有的测试组件都会被包含在测试计划中。测试计划中可以配置被整个测试计划中的组件调用的自定义变量、线程组运行模式、测试中使用的库文件等。

- 在测试计划中使用多种测试组件，就可以构建丰富的测试场景。JMeter中的测试组件大致有以下几类：**线程组、取样器、逻辑控制器、监听器、配置元件、断言、定时器、前置处理器、后置处理器。**

3.1 线程组

- 线程组元件是所有**测试计划的入口**。所有的取样器和控制器必须放在线程组下。一个线程组可以看作一个虚拟用户池，其中的每个线程都可以理解为一个虚拟用户，多个虚拟用户同时去执行相同的一批次任务。每个线程之间都是隔离的，互不影响的。



线程组配置

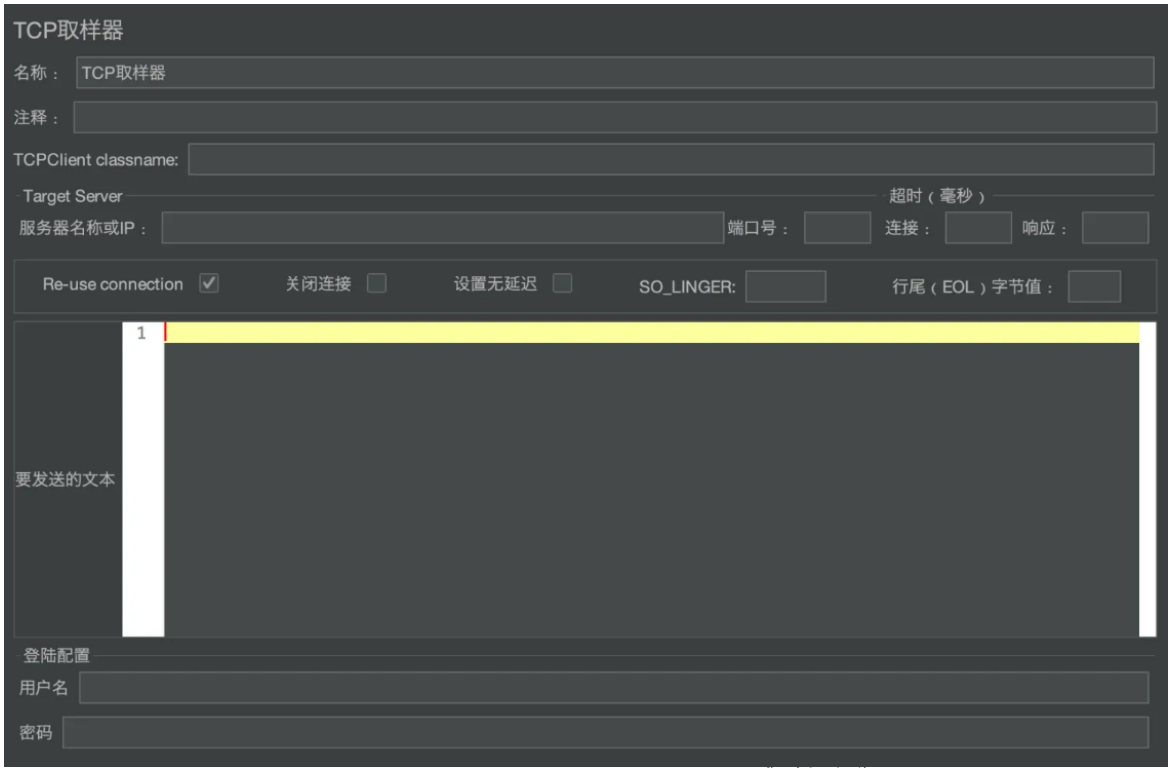
- 取样器错误后要执行的动作
  - 控制了「当遇到错误的时候测试的执行策略」是否会继续执行
- 线程数
  - 线程数也就是**并发用户数**，每个线程将会完全独立地运行测试计划，互不干扰。测试中使用多个线程用于模仿对服务器的并发访问。
- ramp-up时间
  - ramp-up时间用于设置启动所有线程所需要的时间。例如：线程数设置为10，ramp-up时间设置为100秒，则使用100秒启动10个线程，每个线程间隔10秒启动。
  - 如果ramp-up值设置得很小、线程数设置得很大，执行测试时会对服务器产生很大的压力
- 循环次数
  - 设置结束前线程组中每个线程循环的次数。
- 延迟创建线程直到需要
  - 默认情况下，测试开始的时候，所有线程就全部创建了。如果勾选了此选项，那么线程在需要用到的时候才创建。
- 线程组调度器
  - 调度器配置可以更灵活地控制线程组执行的时间
  - （1）持续时间：控制测试执行的持续时间，以秒为单位。
  - （2）启动延迟：控制测试在多久后启动执行，以秒为单位。

3.2 取样器

- 取样器是用来模拟用户操作的，是向服务器发送请求、接收服务器响应数据的运行单元。取样器是包含在线程组内部的组件，因此它必须在线程组中添加。JMeter原生支持多种不同的取样器，如TCP取样器、HTTP请求、FTP请求、JDBC请求、Java请求等，每一种不同类型的取样器根据设置的参数向服务器发出不同类型的请求。
- **建议：使用Badboy或者使用JMeter自带的HTTP代理服务抓取请求。**

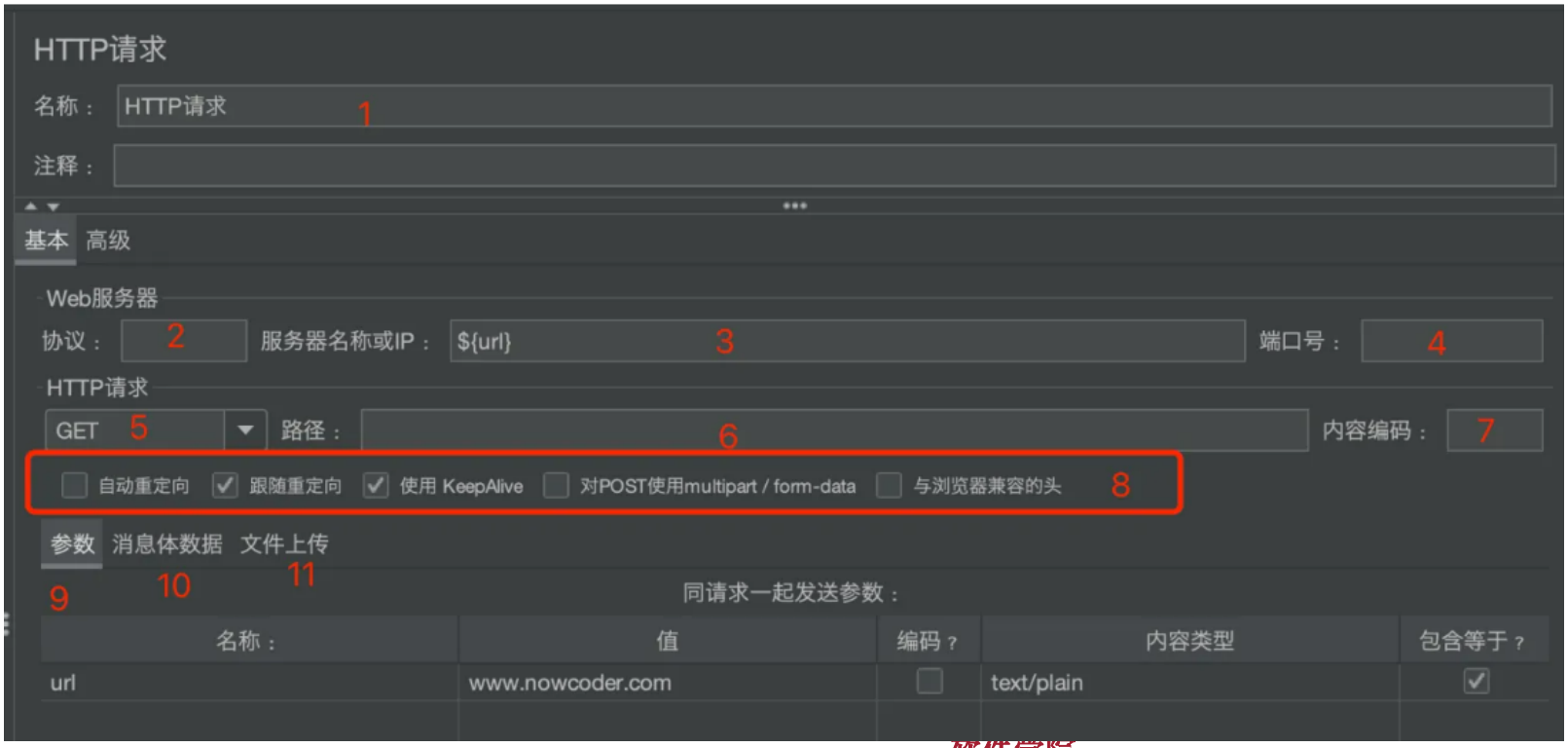
TCP取样器

- TCP 取样器通过TCP/IP来连接指定服务器，连接成功后向服务器发送消息，然后等待服务器回复。
- 可设置的属性：TCPClient classname、目标服务器设置、连接选项、超时时间、要发送的文本、登陆配置



HTTP取样器

- HTTP取样器向web服务器发送HTTP/HTTPS请求。
- 可设置的属性：名称和注释、请求协议、域名或IP地址、端口号、请求方法、路径、内容编码、更多请求选项、参数、消息体数据、文件上传

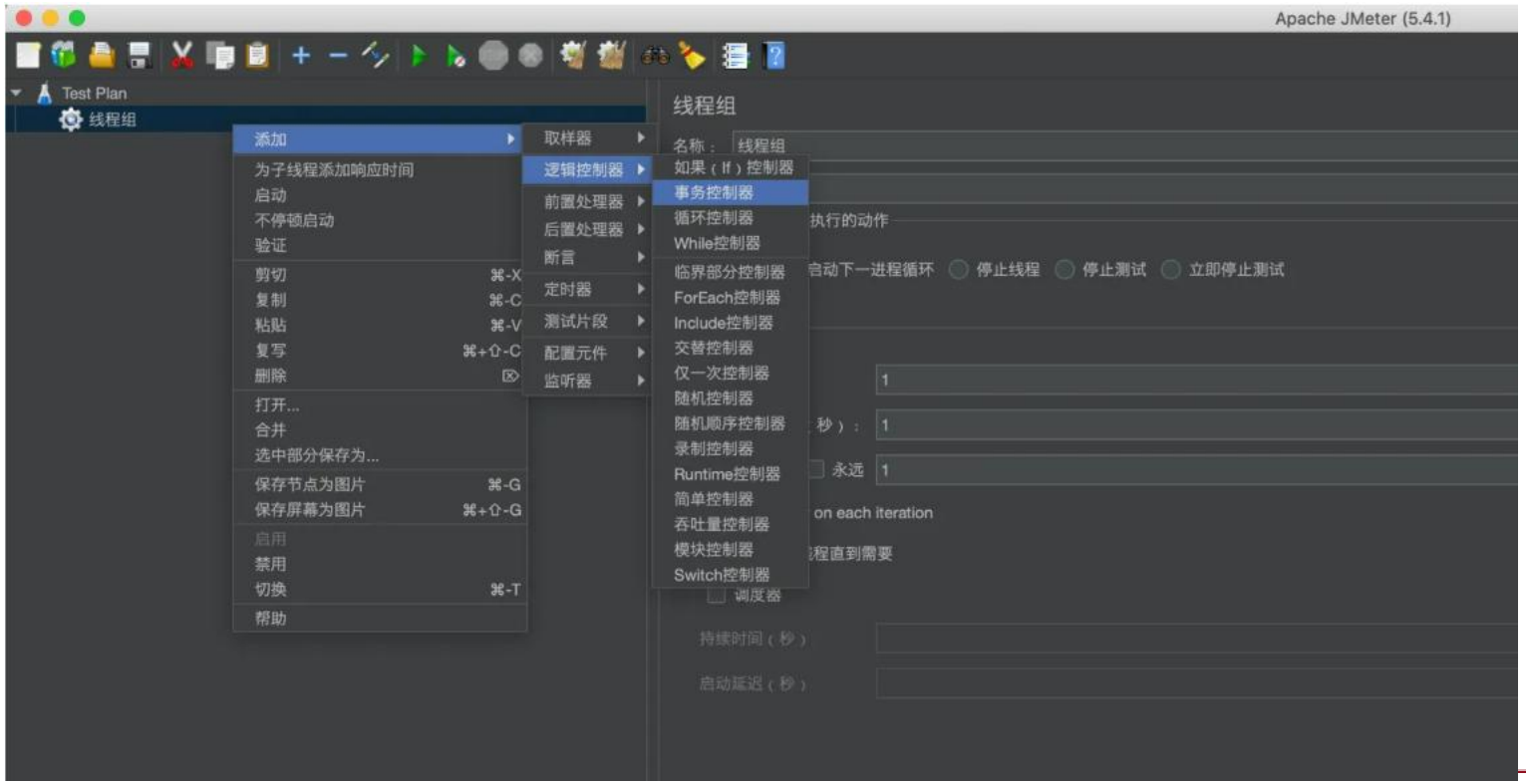


3.3 逻辑控制器

- JMeter 逻辑控制器可以对元件的执行逻辑进行控制，JMeter 官网是这样解释的：「Logic Controllers determine the order in which Samplers are processed」。也就是说逻辑控制器可以控制采样器(samplers)的执行顺序，因此控制器需要和采样器一起使用。除仅一次控制器外，其他逻辑控制器可以相互嵌套。
- JMeter 中的逻辑控制器主要分为两类：
  - 控制测试计划执行过程中节点的逻辑执行顺序，如：循环控制器、If 控制器等；
  - 对测试计划中的脚本进行分组、方便 JMeter 统计执行结果以及进行脚本的运行时控制等，如：吞吐量控制器、事务控制器。

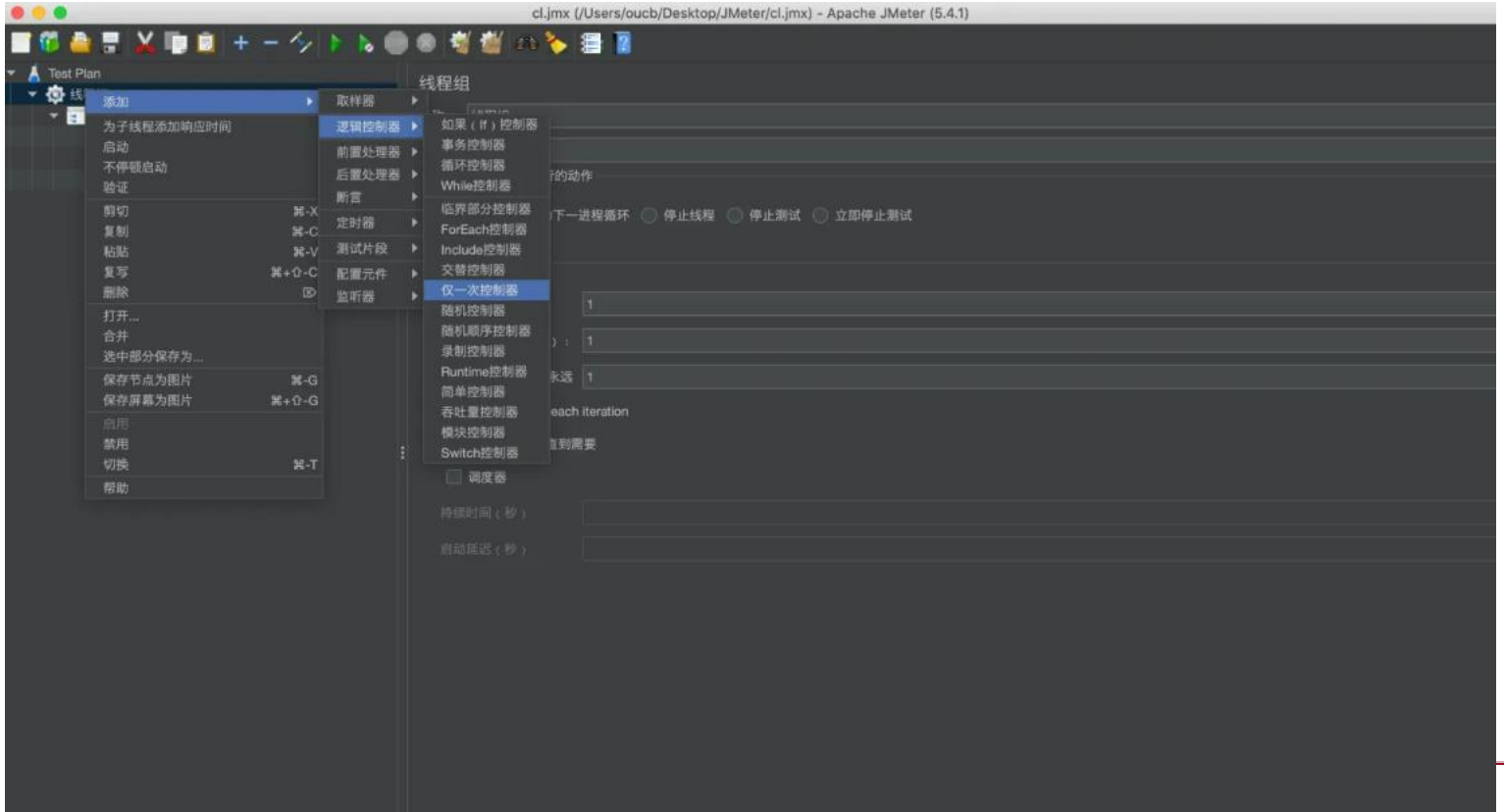
事务控制器

- 统计一组相关请求的的整体响应时间
- 事务控制器会对该控制器下所有子节点的取样器执行消耗时间进行统计。如果事务控制器下定义了多个取样器，所有取样器都运行成功时，整个事务才能算成功。



仅一次控制器

- 只执行一次的控制器，在线程组下的循环执行过程中对该控制器下的请求只执行一次。对于需要登录的测试，可以考虑将登录请求放在仅一次控制器中，因为登录请求只需执行一次即可建立会话。



3.4 监听器

- 监听器是用于对测试结果数据进行处理和可视化展示的一系列元件。察看结果树、 图形结果、聚合报告等都是我们经常用到的监听器组件。
- 察看结果树：以树形结构展示了每一个取样器的结果、请求内容、响应时间、响应码、响应内容等信息



3.5 配置元件

- 配置元件用于提供对静态数据配置的支持。它可以定义在测试计划层级下，也可以定义在线程组或取样器层级下，定义在不同层级，作用域也不同。配置元件主要有用户自定义变量、CSV数据文件设置、TCP取样器配置、HTTP Cookie管理器等。

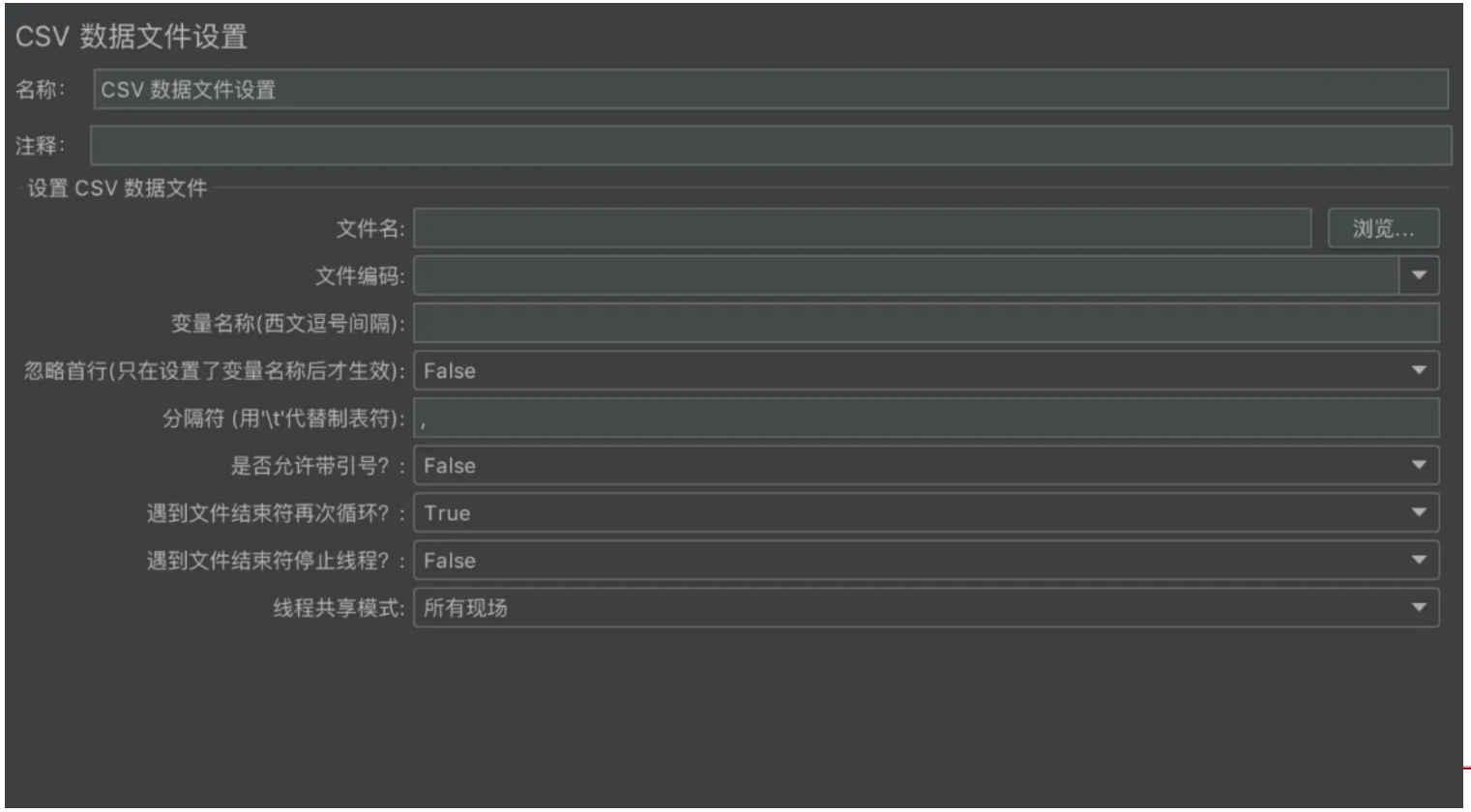
用户自定义变量

- 通过设置一系列的变量，达到在性能测试过程中可以随机选取变量的目的。变量名可以在作用域内引用，通过${变量名}方式来引用变量。
- 除了“用户自定义变量”这个组件外，测试计划和HTTP请求等多个组件中也可以定义变量：



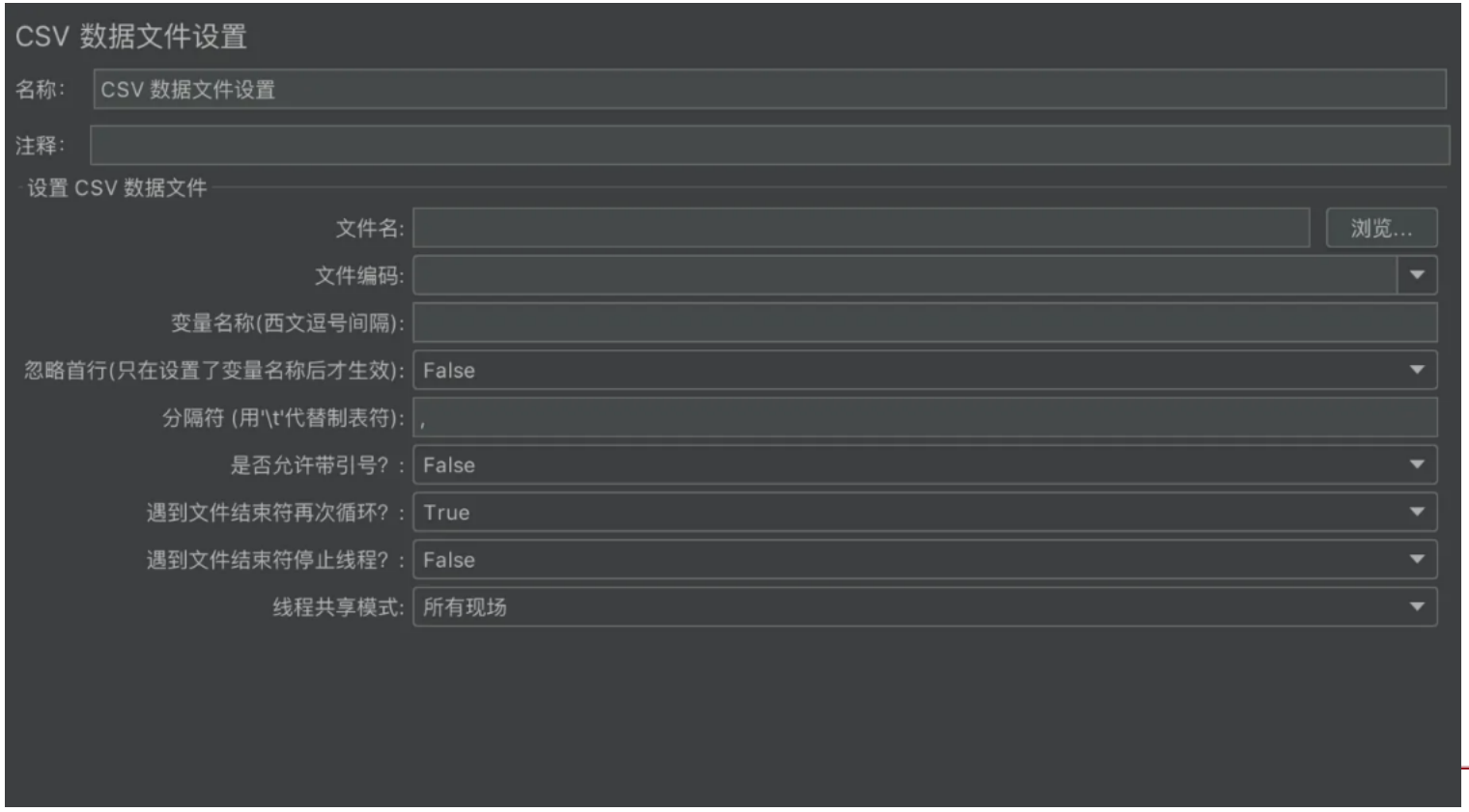
CSV数据文件设置

- 在性能测试过程中我们往往需要一些参数化的输入参数，比如登录操作里面的用户名密码。当并发量比较大的时候，运行时生成数据会对CPU和内存造成较大的负担，而CSV数据文件配置可以作为这种场景下所需的参数来源。



CSV数据文件设置

- 变量名称：定义CSV文件中的参数名，定义后可在脚本在以${变量名}的方式引用
- 遇到文件结束符再次循环：如果设置为True，允许对CSV文件循环取值
- 遇到文件结束符停止线程：如果设置为True，则读取完CSV文件中的记录后停止运行
- 线程共享模式：设置在线程及线程组间共享的模式



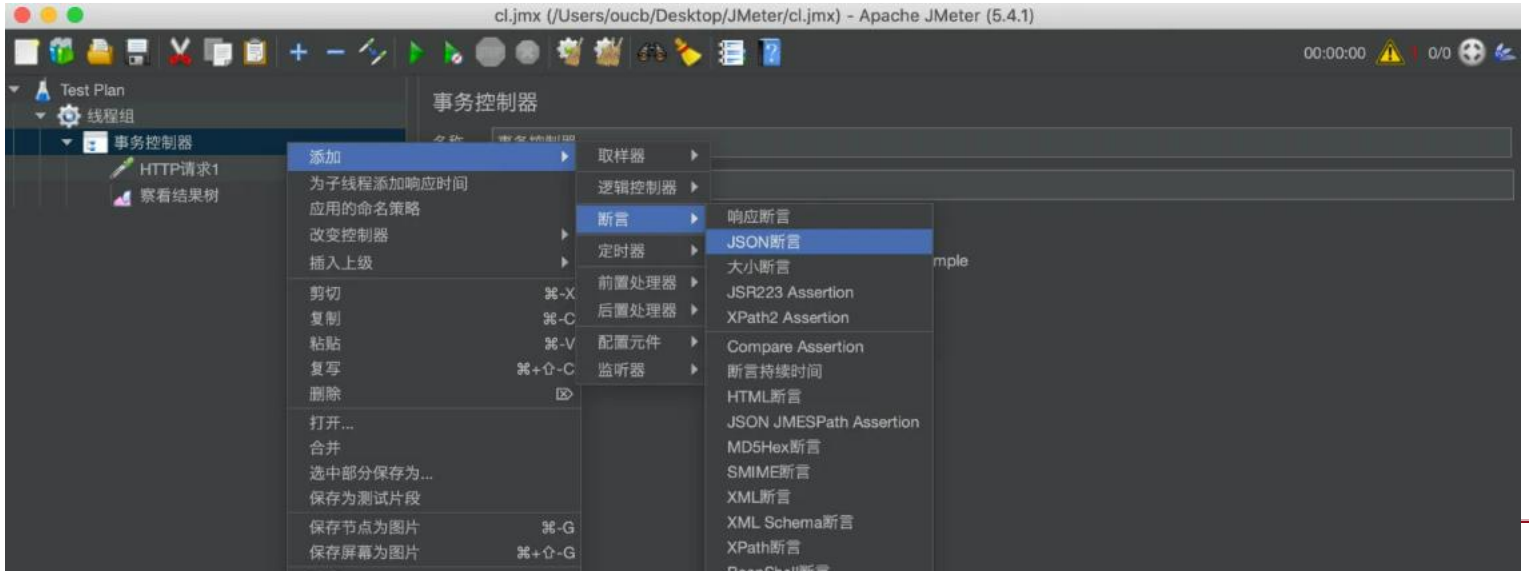
3.6 断言

- 断言即检查接口的返回是否符合预期。断言是自动化测试脚本中举足轻重的一环，因此要十分重视。

- JMeter 常用断言主要有响应断言（Response Assertion）、JSON断言（JSON Assertion）、大小断言（Size Assertion）、断言持续时间（Duration Assertion）、beanshell 断言（Beanshell Assertion）等，这里我们只介绍经常要用到的 JSON断言。

JSON 断言

- JSON断言配置项有：
  - Assert JSON Path exists：需要断言的 JSON 表达式
  - Additionally assert value：如果要根据值去断言，请勾选
  - Match as regular expression：如果要根据正则表达式去断言，请勾选
  - Expected Value：期望值
  - Expect null：如果期望是 null 则勾选
  - Invert assertion：取反



JSON 断言

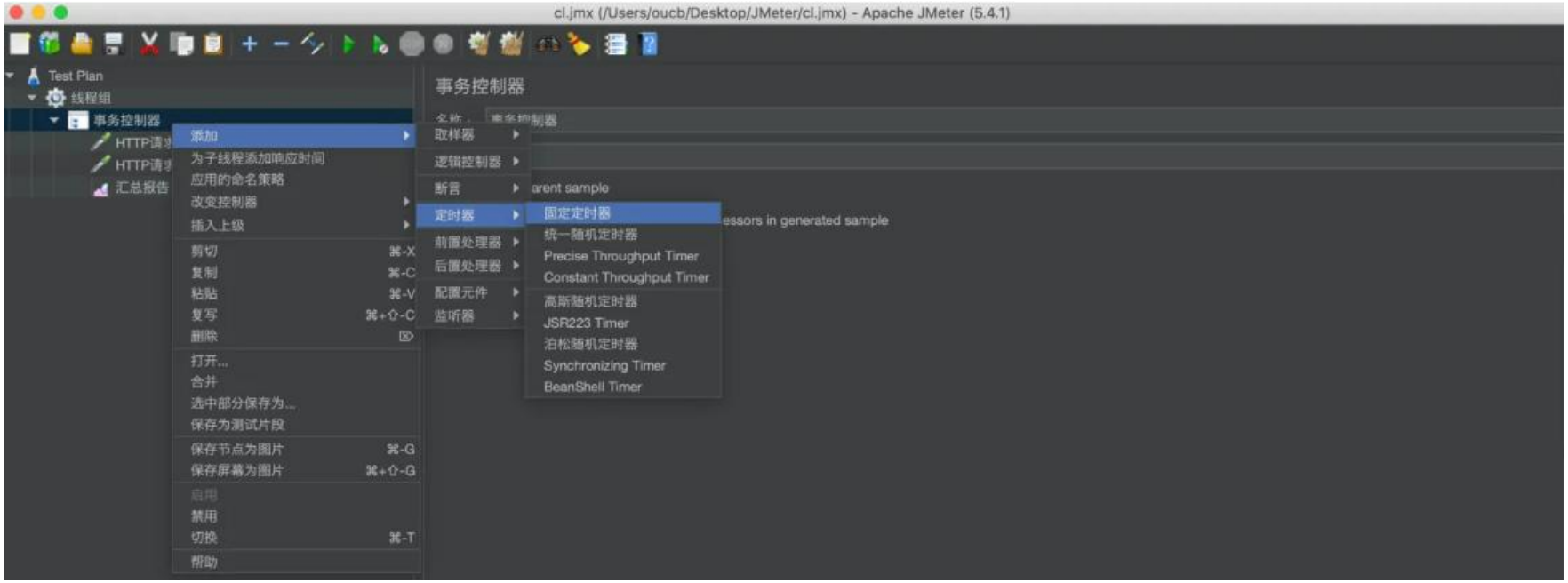
- 断言的判断条件主要包括：
  - 如果响应结果不是 json 格式的，失败；
  - 如果 json path 找不到元素，失败；
  - 如果 json path 找到元素，没有设置条件，通过；
  - 如果 json path 找到元素，但不符合条件，失败；
  - 如果 json path 找到元素，且符合条件，通过；
  - 如果 json path 返回的是一个数组，会迭代判断是否有元素符合条件，有则通过，没有则失败。

3.7 定时器

- 在性能测试中，访问请求之间的停顿时间被称之为思考时间。在实际操作中，停顿时间可以是内容查找、阅读等花费的时间，而定时器正是用来模拟这种停顿时间。其中：
  - 同一作用域下的所有定时器优先于 取样器之前执行。
  - 如果希望定时器仅应用于其中一个取样器，则把定时器加入到该取样器的子节点。
- JMeter定时器主要包括
  - **固定定时器（Constant Timer），**统一随机定时器（Uniform Random Timer），精准吞吐量定时器（Precise Throughput Timer），**常数吞吐量定时器（Constant Throughput Timer），**高斯随机定时器（Gaussian Random Timer），JSR223 定时器（JSR223 Timer），泊松随机定时器（Poisson Random Timer），同步定时器（Synchronizing Timer），BeanShell 脚本编写定时器（BeanShell Timer）。

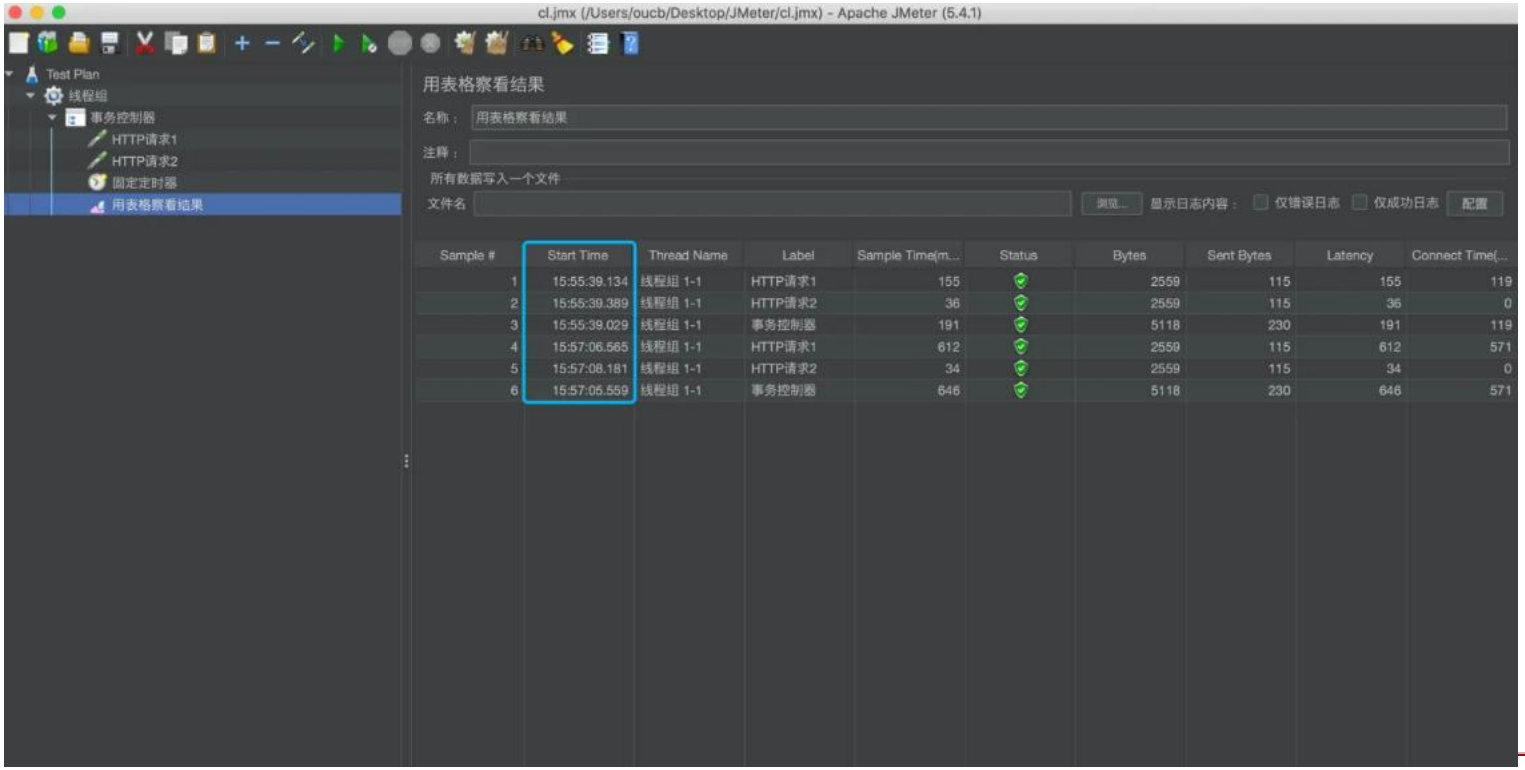
固定定时器

- 固定定时器，即配置每个请求之间的间隔时间为固定值。
- 在一个事务控制器上添加固定定时器：将线程延迟分别配置为 100 和 1000后，运行脚本



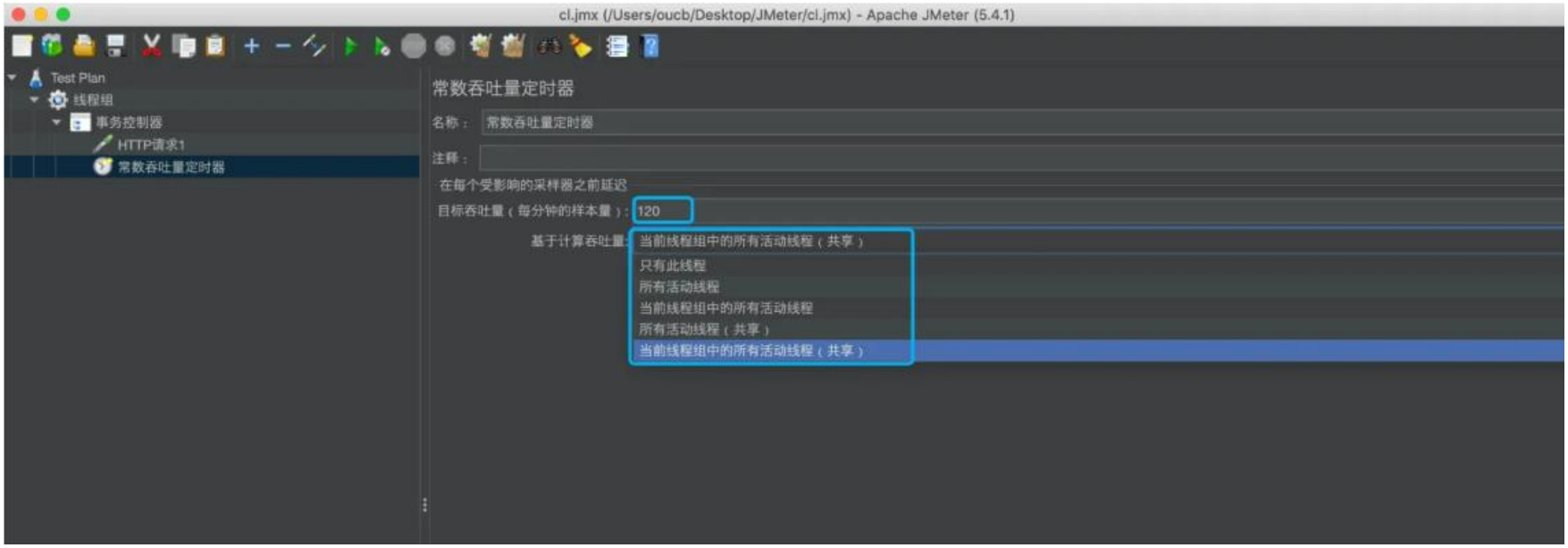
固定定时器

- 在一个事务控制器上添加固定定时器：将线程延迟分别配置为 100 和 1000后，运行脚本
- 查看表格结果中的数据，其中1、2是配置为 100 毫秒时的运行结果，4、5是配置为 1000 毫秒时的运行结果，可看到 4、5 的间隔时间明显比 1、2 的间隔时间长



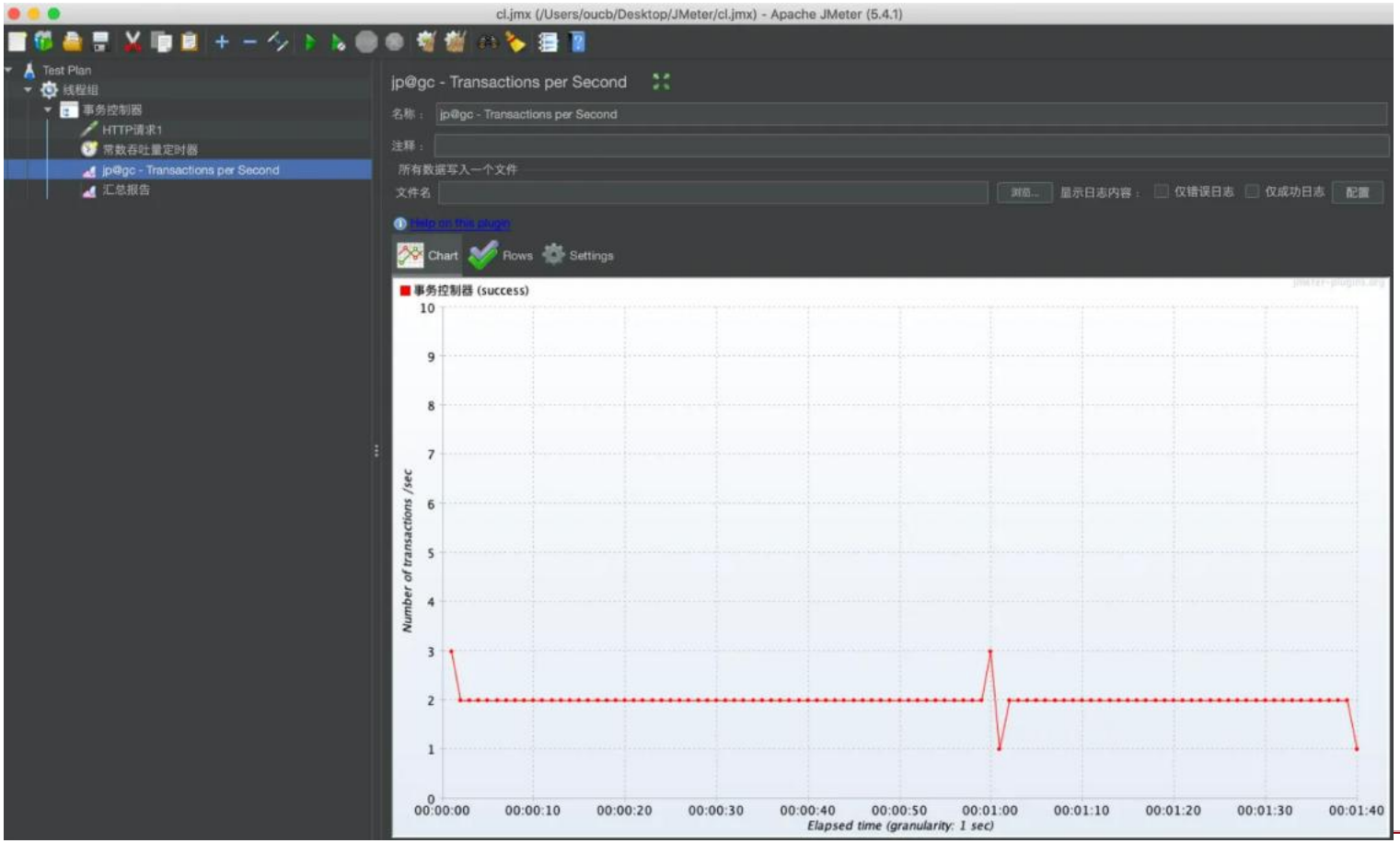
常数吞吐量定时器

- 常数吞吐量定时器用于控制请求按指定的吞吐量去执行。
- 配置目标吞吐量为 120（注意单位是分钟），基于计算吞吐量选择“当前线程组中的所有活动线程（共享）”



常数吞吐量定时器

- 配置目标吞吐量为 120（注意单位是分钟），基于计算吞吐量选择“当前线程组中的所有活动线程（共享）”
- 运行脚本，查看结果，可看到吞吐量基本维持在 2/每秒（120/60）



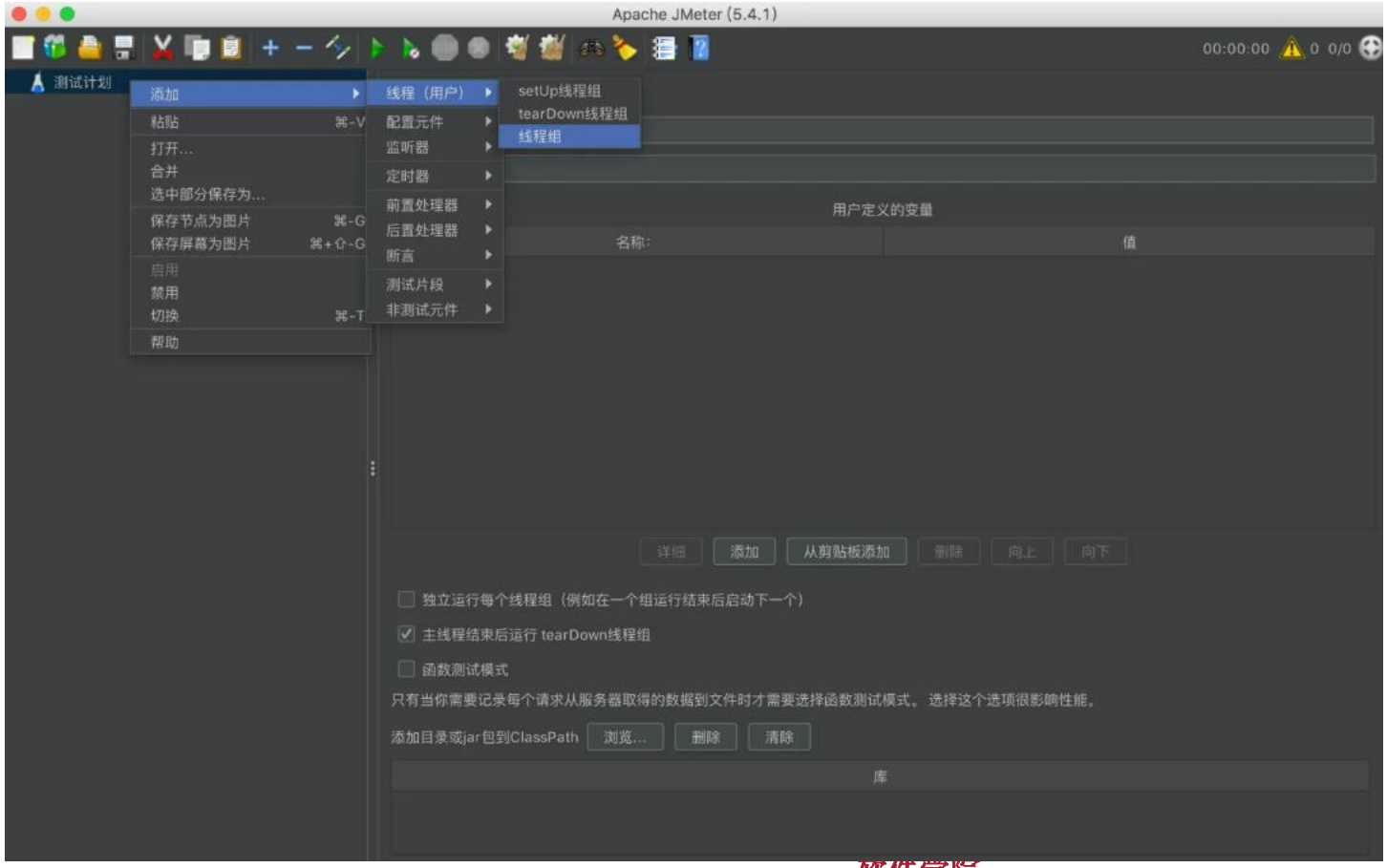
3.8 前置处理器和后置处理器

- 前置处理器是取样器请求之前执行一些操作，经常用于在取样器请求运行前修改参数，设置环境变量，或更新未从响应文本中提取的变量。
- 后置处理器是在取样器请求之后执行一些操作。有时候服务器的响应数据在后续请求中需要用到，我们就需要对这些响应数据进行处理。比如获取响应中的jwt token，在后续请求中使用以进行身份验证，这时就会使用后置处理器。
- 后置处理器可用于构建**关联(Correlation)的测试用例**

4 JMeter测试示例

1. 添加虚拟用户组（Thread Group）

右击测试计划 > 添加 > 线程（用户） > 线程组



4 JMeter测试示例

1. 添加虚拟用户组（Thread Group）

JMeter 使用单个线程来模拟一个用户，用户组 Thread Group 就是指一组用户，作为模拟访问被测系统的虚拟用户组。

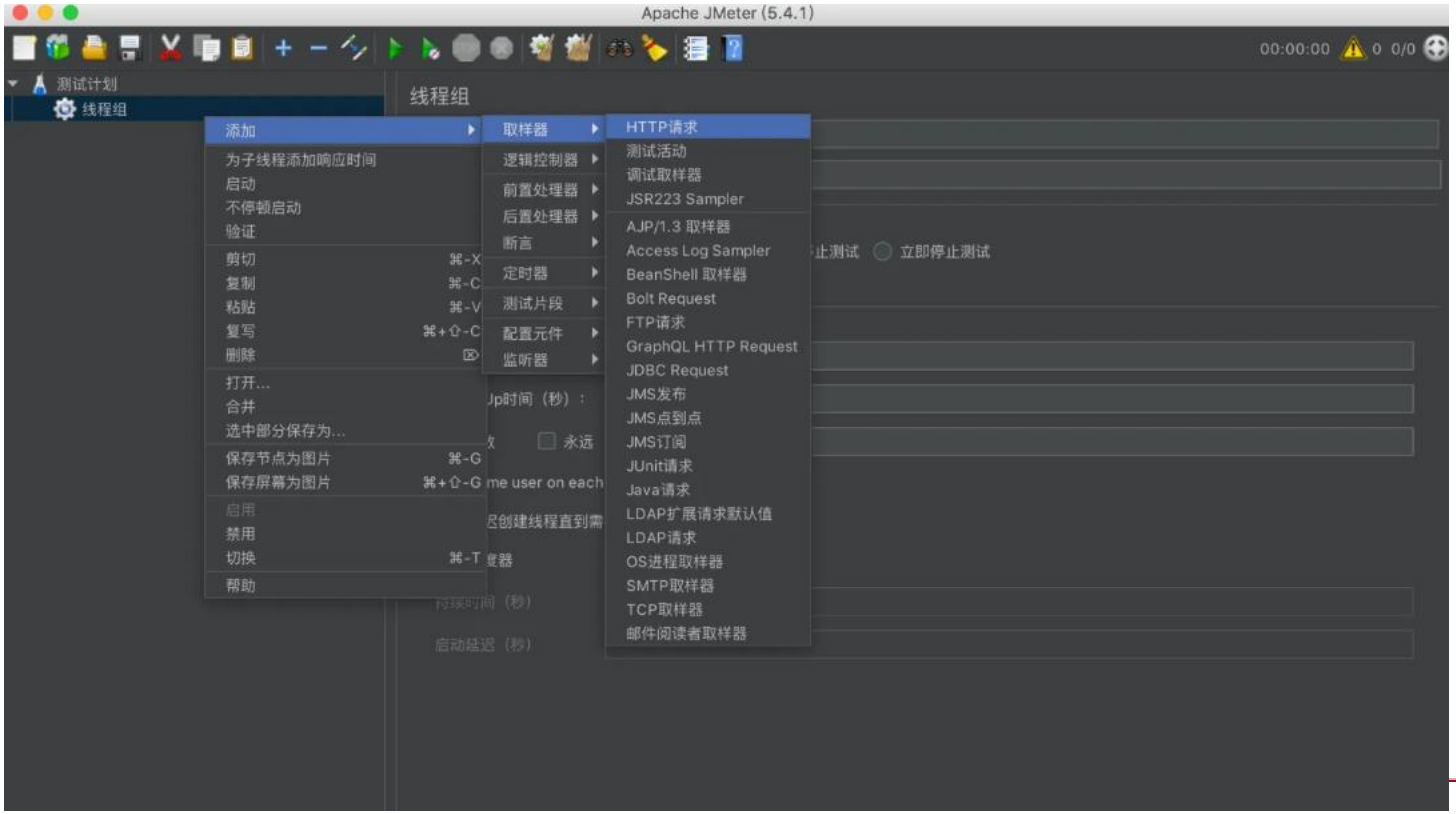
「线程属性」中的「线程数」可用于配置虚拟用户组的并发用户数，数值越高，并发量越大；「循环次数」可用于配置每个虚拟用户执行多少次的测试。



4 JMeter测试示例

2. 添加被测 HTTP 页面

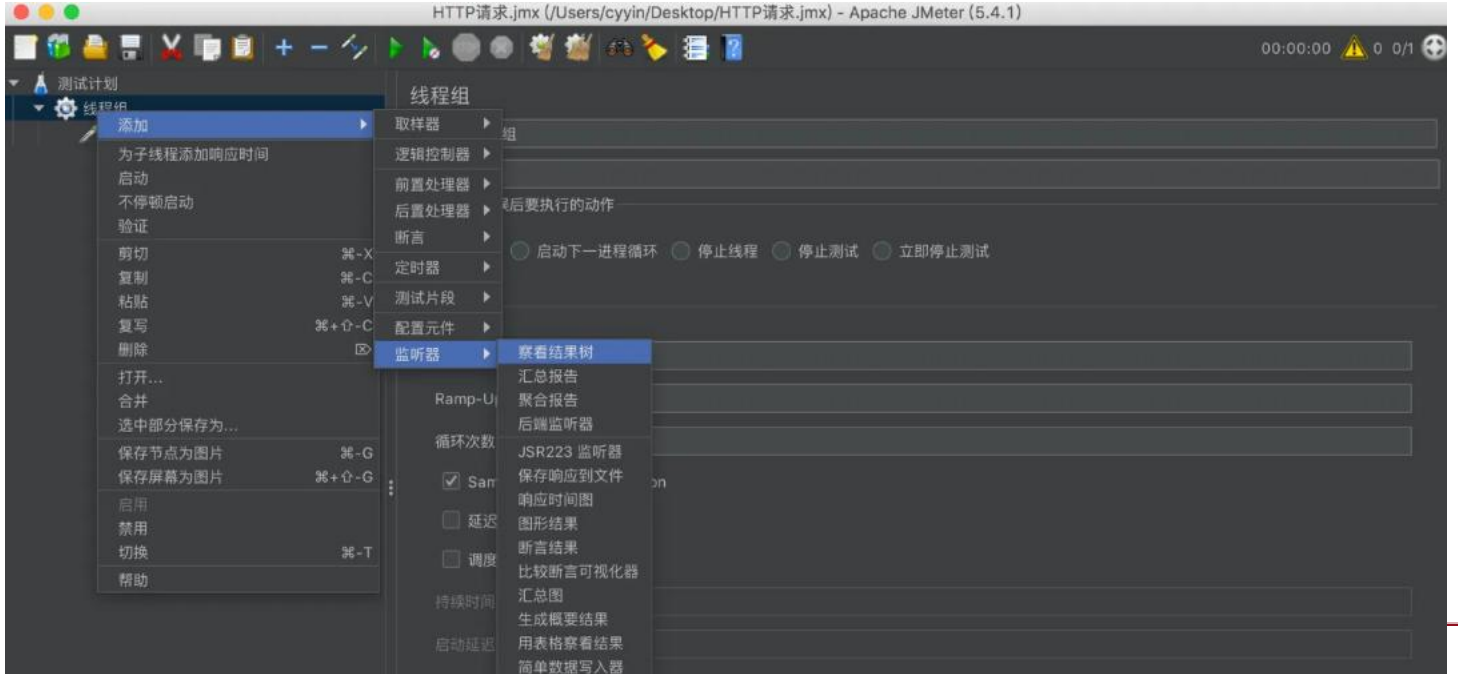
右击线程组 > 添加 > 取样器 > HTTP请求JMeter 使用单个线程来模拟一个用户，用户组 Thread Group 就是指一组用户，作为模拟访问被测系统的虚拟用户组。



4 JMeter测试示例

- 3. 添加结果监听器：右击线程组 > 添加 > 监听器 > 察看结果树

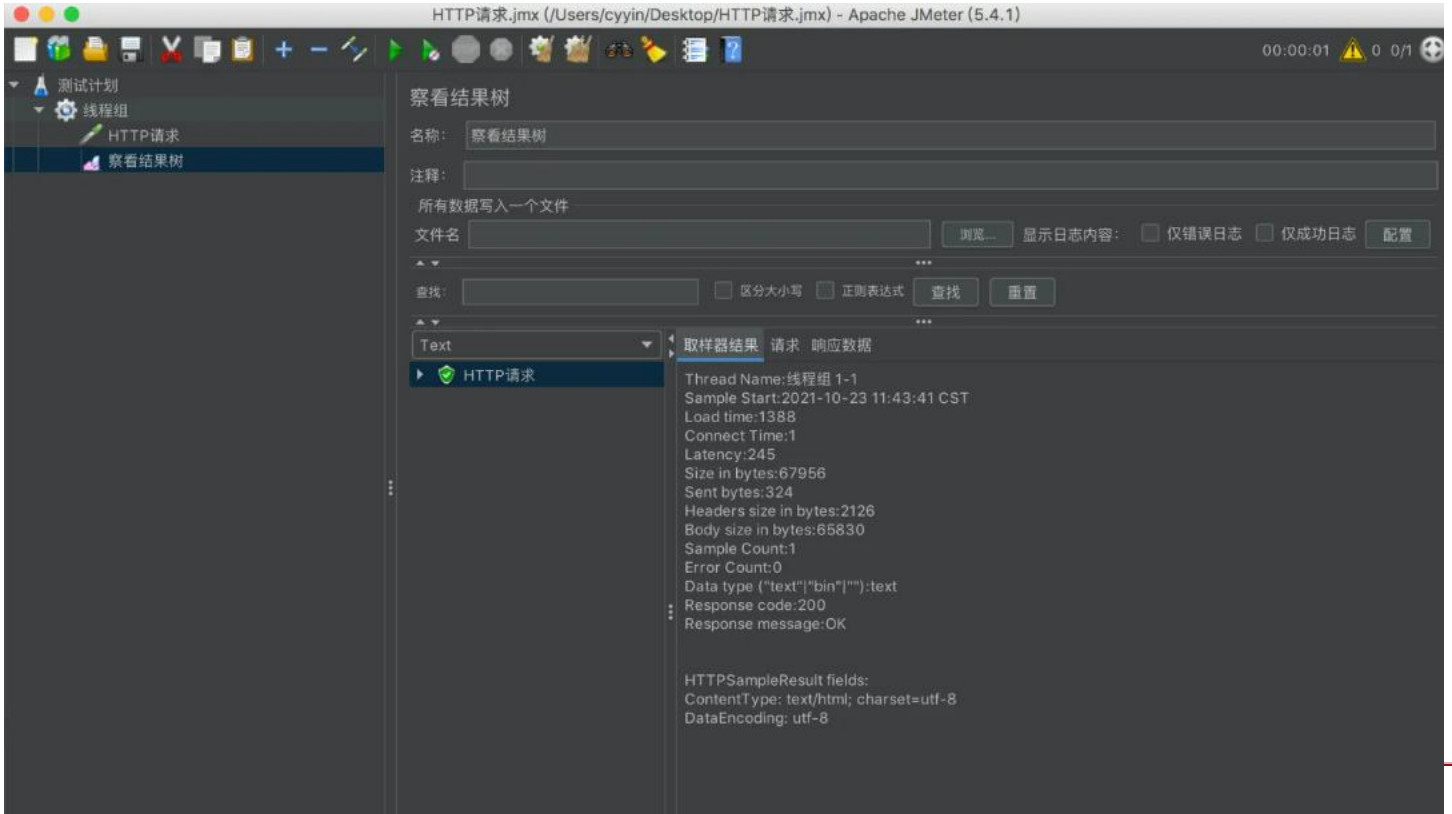
监听器在实际运行性能测试中并不是必须的，但在编写脚本的过程中可以帮助直观看到测试结果，方便调试。在这个样例脚本中我们将使用「察看结果树」来帮助查看请求的响应信息。



4 JMeter测试示例

4. 运行测试

保存测试脚本后，点击操作栏中的「启动」按钮，就开始运行测试脚本了。建议线程组中的线程数和循环次数设置得小一些（比如10以内），以免被 ban。



5 脚本录制工具-Badboy/代理服务器

- **Badboy（Windows only）**

Badboy是一款免费的网页自动化测试工具，它可以录制用户在浏览器中的操作，并生成Jemeter可以识别的.jmx后缀的脚本文件。Badboy的录制方式主要有两种：Request方式和Navigation方式。由于Jemeter对脚本的需求，我们通常使用Request方式进行录制。

- **代理服务器**

Jemeter还提供了自带的代理服务器录制功能。这种方式需要在浏览器中设置代理服务器，将所有请求通过Jemeter代理服务器进行转发，从而录制脚本。

5.1 Badboy

- **安装与设置**

首先，从官方网站下载并安装Badboy。安装过程非常简单，只需双击exe程序即可。安装完成后，打开Badboy，并在工具栏上找到’ N’ 按钮，切换到Request录制模式。

- **脚本录制**

在Request模式下，Badboy会模拟浏览器发送表单信息到服务器，每个资源都将作为请求发送。开始录制前，确保已设置正确的代理服务器和端口（默认为8080）。然后，在Badboy中打开需要录制的网页，并进行相应的操作。Badboy会自动记录这些操作，并生成Jmeter脚本。

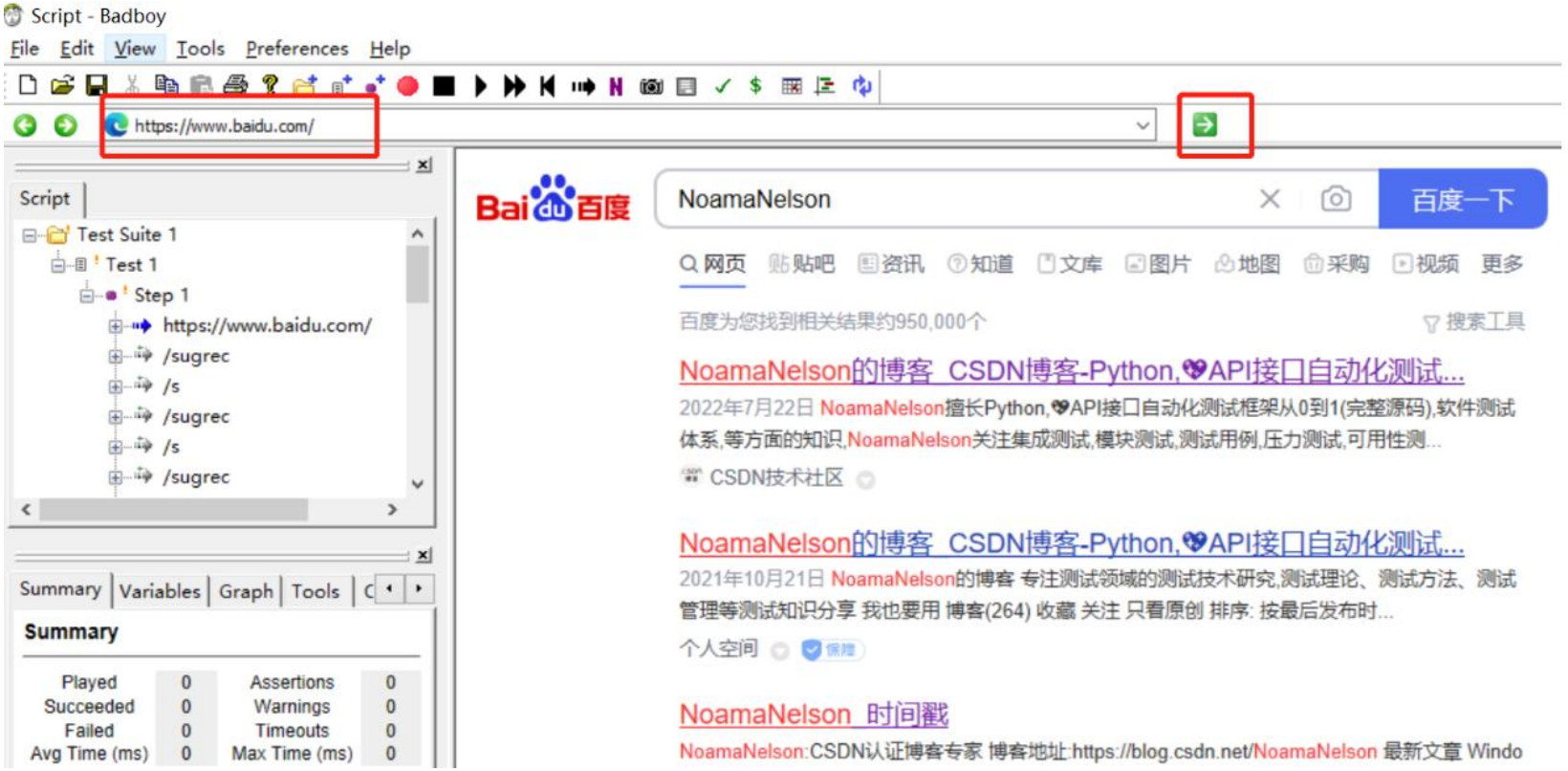
5.1 Badboy

- **脚本导出**

录制完成后，可以在Badboy中选择导出脚本。导出的脚本将保存为**.jmx**格式，这是Jmeter脚本的标准格式。导出的脚本可以直接在Jmeter中打开和编辑。

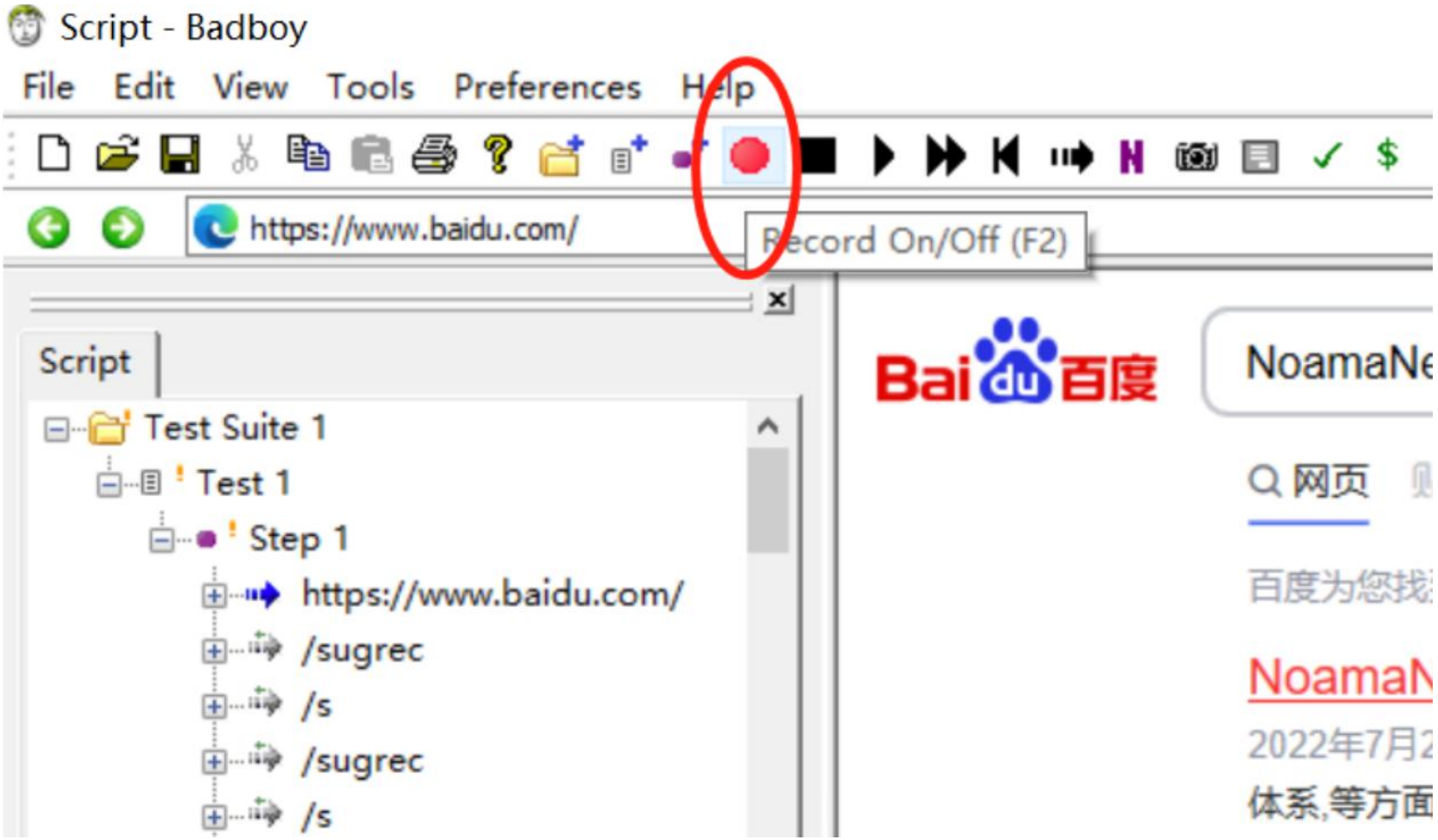
5.1 Badboy

地址栏输入：https://www.baidu.com/，点后边的箭头进行录制



5.1 Badboy

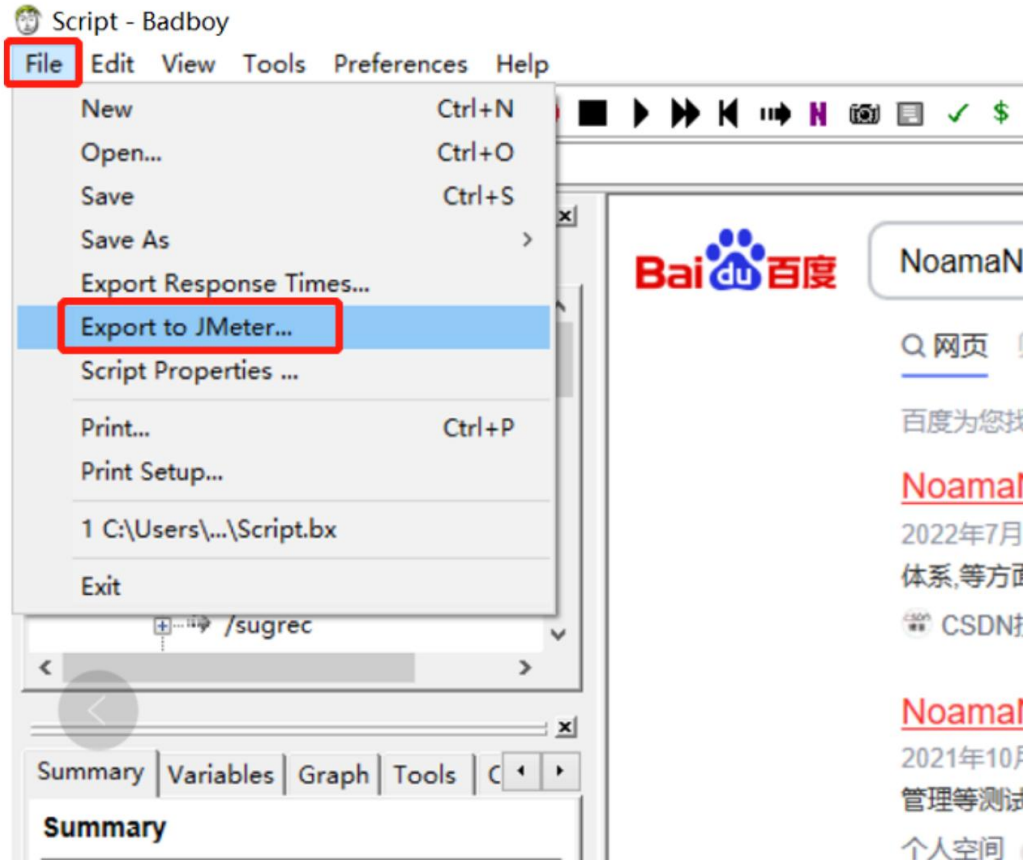
默认打开工具是录制状态



5.1 Badboy

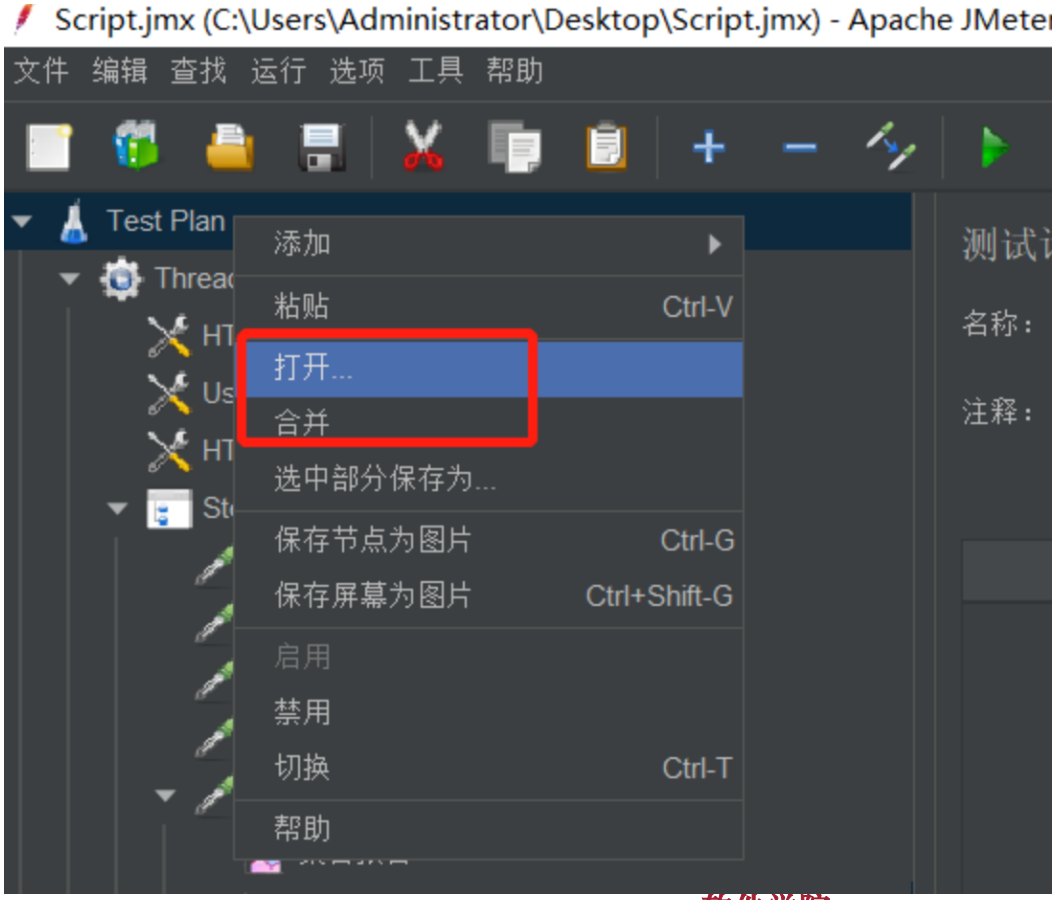
输入 NoamaNelson，进行搜索后，停止录制

导出脚本为 Jmeter 可识别的脚本即可，保存为：Script.jmx



5.1 Badboy

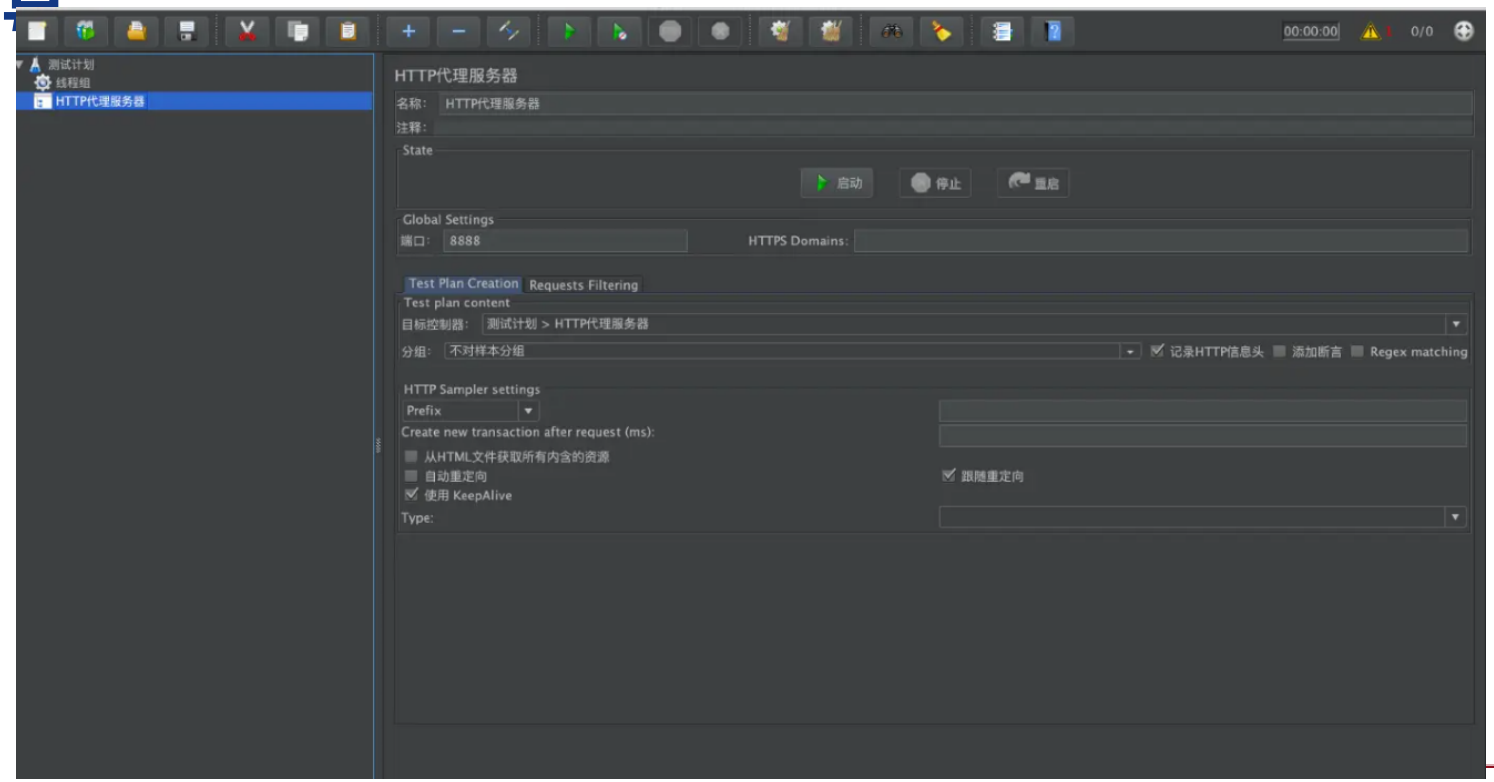
导入脚本，测试计划-右键”打开“，选择刚才导出的Script.jmx



5.2 代理服务器

设置代理服务器

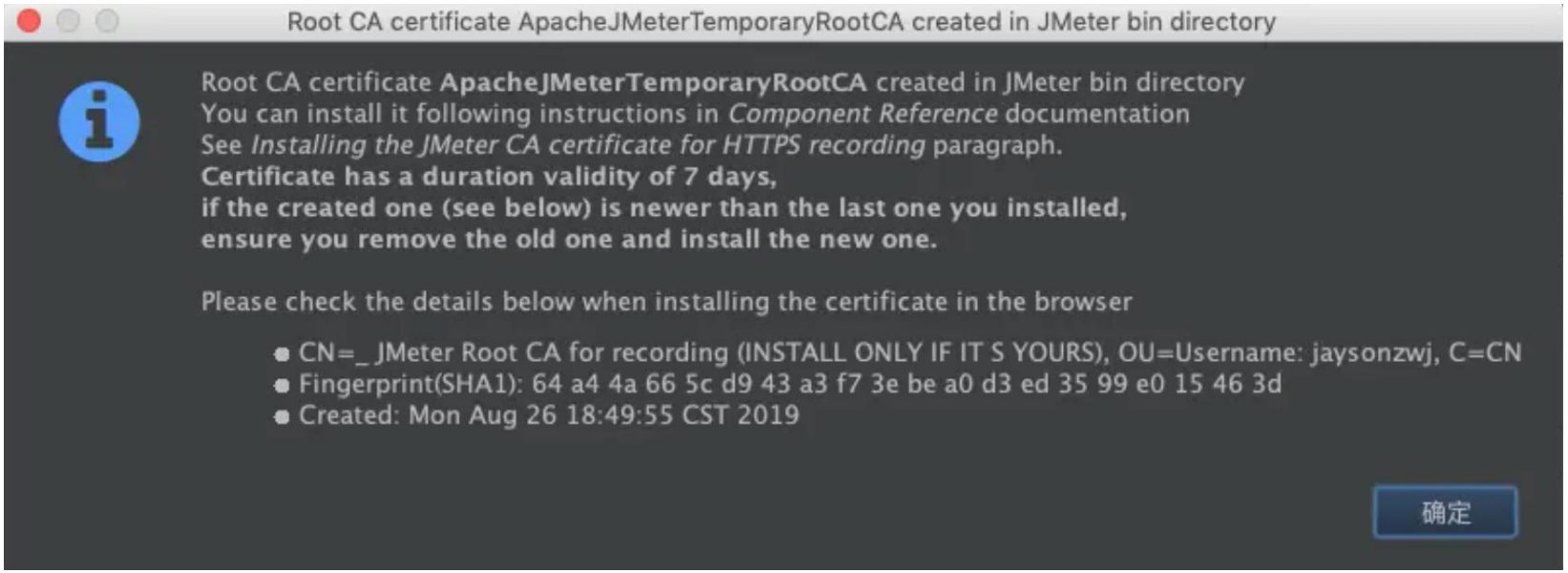
在Jemeter中，添加一个线程组，并设置相应的参数，如线程数、Ramp-up periods和循环次数等。然后，在Jemeter中启动代理服务器，并记录下代理服务器的**端口号**



5.2 代理服务器

保存Jmeter证书文件

点击“启动”按钮后出现https提示，点击确定生成证书文件（文件在Jmeter文件夹的bin目录下）



5.2 代理服务器

浏览器设置

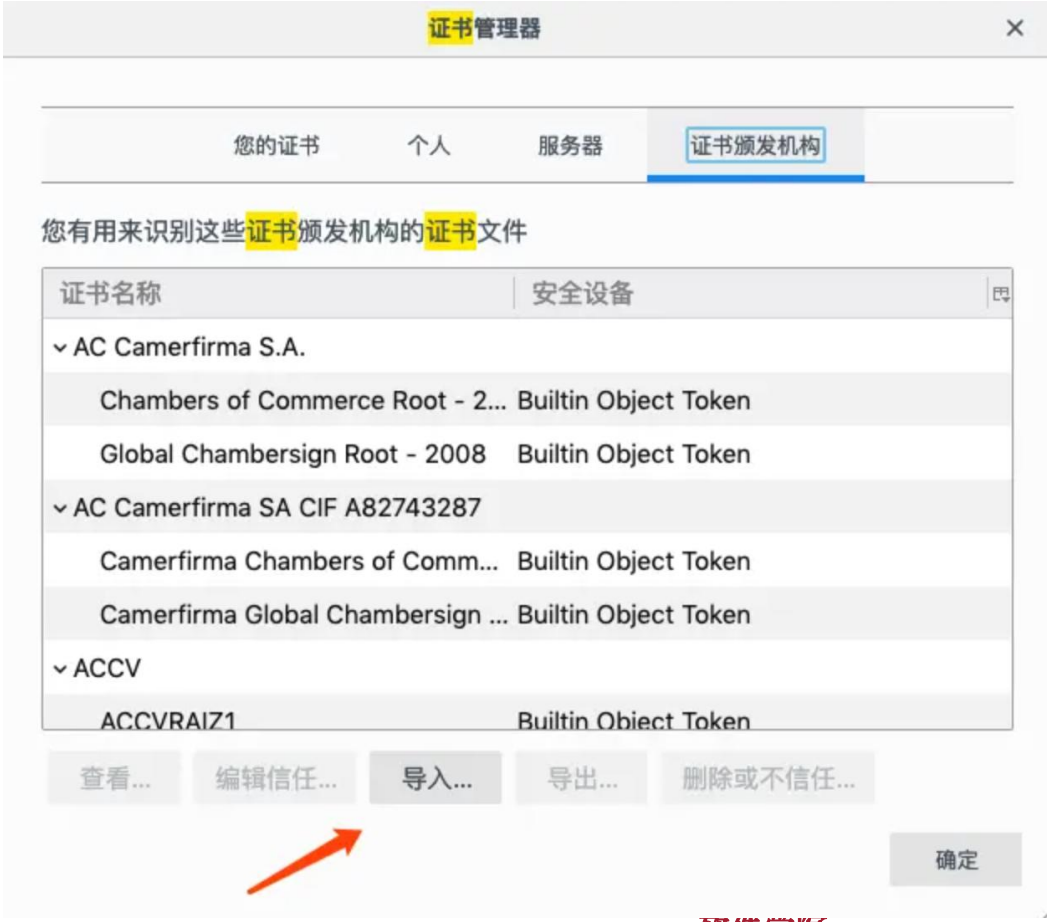
打开需要录制脚本的浏览器，并设置其代理服务器为Jemeter的代理服务器。确保代理服务器的端口与Jemeter中设置的端口一致。



5.2 代理服务器

- 浏览器添加信任证书

访问https请求前需要先导入jmeter生成的证书



5.2 代理服务器

- **脚本录制**

设置好代理服务器后，开始在浏览器中进行相应的操作。这些操作将被Jemeter代理服务器记录下来，并生成相应的脚本。录制完成后，可以在Jemeter中查看和编辑生成的脚本。

- **脚本保存**

生成的脚本将自动保存在Jemeter中。可以根据需要选择保存脚本为.jmx格式，以便后续使用。

6 参考文档

1. https://jmeter.apache.org/
2. https://riptutorial.com/jmeter/example/27414/script-recording-with-badboy

复杂系统中性能测试场景

- 各种OS软件，中间件，复杂网络环境中的性能测试
  - IPC、RPC效率
  - 网络交换协议、网络拓扑结构、网络传输速率
- 计算任务、人工智能任务、大模型任务的性能测试
  - 算力与时间直接的关系
  - 精度与效率之间的关系
  - 大模型的TPS、TTFT、TBT等
- 数据库、数据访问、存储系统、文件系统中的性能测试
  - 数据访问查询QPS，时延，尾部时延
  - 数据访问的数学分布
  - YCSB等典型benchmark

总结与展望

- **使用大模型等AI应用测试Web应用**
- **智能测试用例生成**

  使用LLM解析用户故事/需求文档，生成可执行测试脚本

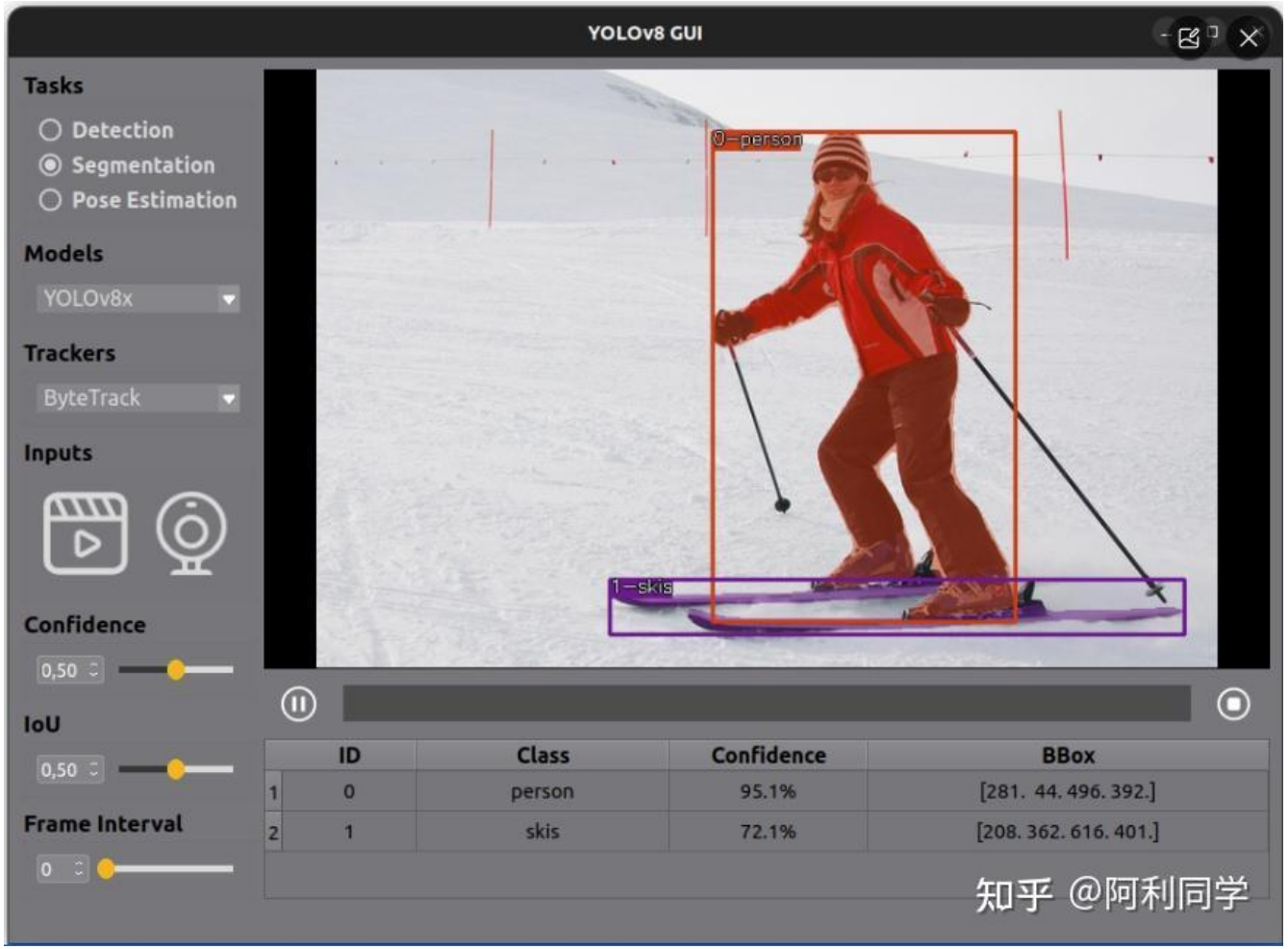
  结合代码静态分析（AST）生成边界条件测试
- **视觉回归测试增强GUI测试**

  目标检测：YOLOv8定位页面元素

  相似度计算：CLIP模型验证图文语义一致性
- **性能测试（压力测试）**

  自动化生成大量用户数据模拟压力测试

  基于历史数据预测web软件接口的极限QPS，提升性能测试准确率


YOLOv8 GUI
Tasks
Detection
Segmentation
Pose Estimation
Models
YOLOv8x
Trackers
ByteTrack
Inputs
Confidence
0,50
IoU
0,50
Frame Interval
0
0-person
1-skis
ID
Class
Confidence
BBox
1
0
person
95.1%
[281. 44. 496. 392.]
2
1
skis
72.1%
[208. 362. 616. 401.]
知乎 @阿利同学



```
from selenium import webdriver
from ultralytics import YOLO
import cv2
import numpy as np

# 设置Selenium驱动并打开网页
driver = webdriver.Chrome()
driver.get('http://example.com')
driver.save_screenshot('screenshot.png')
driver.quit()

# 使用YOLOv8进行图像处理和检测
model = YOLO('runs/detect/train/weights/best.pt')
results = model('screenshot.png')

# 处理结果（例如，打印检测到的对象）
for result in results:
    boxes = result.boxes  # 获取检测到的框信息
    for box in boxes:
        print(f"Detected object: {box.cls} at {box.xyxy}")  # 打印类别和框坐标
```